Berliner Hochschule für Technik

Department VII

M.Sc. Information and Communication Engineering

Master's Thesis

Integrated Edge-to-Cloud Systems: Federated Learning for Sensor Fusion and Object Recognition

submitted by

Adnan Anwar Rajput

Matriculation number 931043

First supervisor: Prof. Dr. André Jakob

Second supervisor: Prof. Dr. Marcus Purat

Berlin, October 2026

Task Description (Aufgabenstellung)

Task as agreed with the supervisor, Prof. Dr.-Ing. André Jakob, in the exposé of 23 February 2026.

Topic: Integrated Edge-to-Cloud Systems: Federated Learning for Sensor Fusion and Object Recognition

Problem. Camera-only robotic systems fail in low-light or visually obstructed environments. Adding audio sensors improves perception, but raises the question of where and how the machine learning should happen, balancing privacy, power consumption and model accuracy.

Task. Compare three learning strategies for a multimodal edge system and, based on the comparison, recommend a suitable strategy:

1. Develop a multimodal edge system that integrates a camera and a microphone on Raspberry Pi edge devices connected to a server.
2. Implement three strategies: centralized learning (raw data sent to the server for training), hybrid learning (spectrograms computed on the device and sent to the server) and federated learning (local training on the device, only model weights shared).
3. Design an audio processing pipeline based on the short-time Fourier transform (STFT) on the device, use a pre-trained vision model (YOLOv8) to label the audio automatically, and develop a model that combines visual and auditory inputs for object recognition.
4. Evaluate and compare the strategies by accuracy, privacy, power consumption, bandwidth, latency and convergence behaviour.
5. Select and justify the most suitable strategy based on the results.

Abstract

Robots and autonomous systems mostly rely on cameras to recognise objects. In fog, storms or darkness, however, the camera has difficulty detecting objects accurately. In such situations, sound can take over, because objects can also be recognised by the sound they make. To learn these sounds, the system uses the camera itself as a teacher: while the camera can see, an object detector (YOLOv8) names the object, and the sound recorded at the same moment receives that name. In addition, sound and camera images are combined in a fusion model, so that the system uses both senses when the camera works and still recognises objects by sound when it fails.

In this thesis, different strategies for training such a system were implemented and compared, to observe which one gives the best results and what the trade-offs are. The strategies differ in where the training takes place: on a central server (centralized), partly on the device, which converts the sound into spectrograms (hybrid), or completely on the device, which sends only model weights (federated). In all three, the trained model makes its decisions on the device. They were implemented with a Raspberry Pi edge device, two simulated edge devices and a laptop server communicating over MQTT, and compared by accuracy, upload bandwidth, workload on device and server, latency and privacy.

Centralized training was the easiest to implement and keeps the device light, but raw audio leaves the device. The hybrid strategy reduced this exposure but saved only about a quarter of the bandwidth. Federated learning needed about 16 times less upload, kept the data on the device and reached similar accuracy, but only after its learning rate was made much smaller. The fusion model kept recognising objects when the camera was blacked out, while a camera-only model failed.

Federated learning is therefore recommended for a system that keeps learning in the field, as it also makes frequent model updates cheap; how fast each strategy adapts to new objects is left for future work. The complete system was also tested live on a Raspberry Pi 5.

Keywords: federated learning, edge computing, sensor fusion, audio classification, cross-modal supervision, MQTT, Raspberry Pi

Table of Contents

1 Introduction 1
1.1 Motivation 1
1.2 Problem Statement 2
1.3 Research Questions and Objectives 3
1.4 Contributions 4
1.5 Structure of the Thesis 4
2 Background and Related Work 6
2.1 Audio Signals and the Spectrogram 6
2.2 Convolutional Neural Networks for Sound 8
2.3 Object Detection as a Teacher: YOLO 9
2.4 Cross-Modal and Weak Supervision 10
2.5 Federated Learning 10
2.5.1 The idea 10
2.5.2 The FedAvg algorithm 11
2.5.3 What actually happens to the weight matrices 12
2.5.4 Why averaging weights works at all 13
2.5.5 Local steps and client drift 13
2.5.6 Communication cost 14
2.5.7 Privacy limits 14
2.5.8 Frameworks 14
2.6 Edge Computing and MQTT 15
2.7 Multimodal Fusion and Modality Imbalance 15
2.8 Research Gap 16
3 System Design and Methodology 17
3.1 Research Approach 17
3.2 System Architecture 17

3.2.1 The edge loop 18
3.2.2 Batching 19
3.3 Communication Design 19
3.4 The Three Strategies 20
3.5 Datasets 22
3.5.1 UrbanSound8K 22
3.5.2 VGGSound 23
3.6 Models 23
3.6.1 AudioCNN 23
3.6.2 Fusion model 24
3.6.3 Placing the fusion model in the strategies 25
3.7 Experimental Protocol 25
3.7.1 Why the sensors are replayed 26
3.8 Metrics 27
3.9 Tools and Use of AI Assistance 27
4 Implementation 28
4.1 Hardware and Software 28
4.2 Audio Pipeline 28
4.3 Labelling Pipeline 29
4.4 Client and Server Logic 29
4.5 Fusion Model Training 30
4.6 Live Deployment on the Raspberry Pi 31
4.7 Problems and How They Were Solved 32
4.8 Order of Implementation 33
5 Results 35
5.1 Building Blocks 35
5.1.1 YOLOv8 as a labeller 35

5.1.2 Baseline AudioCNN 35
5.2 Strategy A: Centralized 36
5.3 Strategy B: Hybrid 37
5.4 Strategy C: Federated and the Learning-Rate Sweep 37
5.5 Comparison of the Three Strategies 39
5.6 Live Validation on the Raspberry Pi 39
5.6.1 Enrollment 39
5.6.2 Strategies A, B and C on the Pi 40
5.6.3 Which training settings work on the live data? 42
5.6.4 Live inference 42
5.7 The Fusion Model and the Blackout Test 43
5.8 Fusion Model under the Hybrid and Federated Strategies 45
5.9 Latency 46
5.9.1 Inference latency 47
5.9.2 Update latency 47
5.10 Checking the Measurements against the Formulas 48
5.10.1 Spectrogram shape 48
5.10.2 Bytes per sample in strategies A and B 49
5.10.3 Upload per round in federated learning 49
5.10.4 When does federated learning save bandwidth? 50
5.10.5 Bytes per sample in the hybrid fusion strategy (FB) 50
5.10.6 Growth of server training time in Strategy A 51
5.10.7 FedAvg weights in the experiments 51
5.10.8 A blackout score that looks like chance 51
5.10.9 Summary 52
6 Discussion 53
6.1 RQ1: How Do the Three Strategies Compare? 53

6.2 RQ2: How Does FedAvg Combine the Weights, and When Does It Match Centralized Training? 54
6.3 RQ3: Does Fusion Survive When the Camera Fails? 55
6.4 Privacy 56
6.5 Comparison with Related Work 56
6.6 Comparison with My Exposé 56
6.6.1 Predictions versus results 56
6.6.2 The five metrics 57
6.6.3 Changes to the plan 58
6.7 Limitations and Threats to Validity 59
6.8 Personal Reflection 60
7 Conclusion and Future Work 62
7.1 Summary 62
7.2 Recommendation 62
7.3 Future Work 63
References 64
Appendices 67
Appendix A — Reproducing the Experiments 67
Appendix B — Full Per-Round Tables 68
Appendix C — Evidence Gallery Samples 70
Appendix D — Source Code 70

List of Figures

Figure 1.1: The core idea: while the camera can see, YOLOv8 labels the sound; when the camera fails, the trained audio model recognises the object from sound alone. 2

Figure 2.1: Three 500 ms clips recorded by the Pi (car, cell phone, person) as waveforms (top) and as log-mel spectrograms (bottom)..... 8

Figure 2.2: Steps from waveform to the 51 × 128 spectrogram. 8

Figure 2.3: One FedAvg round between the server and two edge nodes. 12

Figure 3.1: System architecture with two edge nodes, the MQTT broker and the server. .. 18

Figure 3.2: The edge loop that runs every 500 ms. 19

Figure 3.3: What the edge does, what crosses the network and what the server does in each strategy..... 21

Figure 3.4: Message sequence for strategies A and B. 22

Figure 3.5: Fusion model architecture. 24

Figure 3.6: Placement of the fusion model in the hybrid (FB) and federated (FC) strategies. 25

Figure 4.1: Live pipeline: enrollment on the Pi, replay by the A, B and C clients, training on the unchanged server, and live inference on the Pi. 32

Figure 5.1: Test accuracy (left) and server training time (right) per round for Strategy A. 37

Figure 5.2: Test accuracy per federated round for four local learning rates (dashed: starting model). 38

Figure 5.3: Upload per node for strategies A, B and C. 39

Figure 5.4: Frames from the live enrollment with the label YOLO assigned (top right and bottom left are wrong labels). 40

Figure 5.5: Upload and final accuracy of the three strategies on the real Raspberry Pi 5... 41

Figure 5.6: Clean and blackout accuracy for each model (run 2)..... 43

Figure 5.7: Clean (left) and blackout (right) accuracy per round for FB and FC. 46

List of Tables

Table 1.1: The two models in the system and their roles. 3

Table 2.1: STFT settings. 7

Table 2.2: The weight arrays of the AudioCNN. 13

Table 3.1: MQTT topics. 20

Table 3.2: Division of work in the three strategies. 20

Table 3.3: AudioCNN architecture (111,370 parameters for 10 classes). 23

Table 3.4: Metrics used in the experiments. 27

Table 4.1: Hardware. 28

Table 4.2: Software. 28

Table 4.3: Differences between the replayed client and the Pi client. 31

Table 4.4: Order of implementation. 34

Table 5.1: YOLOv8n as an automatic audio labeller. 35

Table 5.2: Per-class results of the baseline AudioCNN on fold 10. 36

Table 5.3: Strategy A per round. 36

Table 5.4: Strategy B, selected rounds. 37

Table 5.5: Effect of the local learning rate on federated training. 38

Table 5.6: Comparison of strategies A, B and C. 39

Table 5.7: Live run on the Raspberry Pi 5 with 96 training samples and three classes (chance level 0.333). 41

Table 5.8: Test accuracy on the live dataset (32 test clips, chance 0.33), mean ± standard deviation over three seeds. 42

Table 5.9: Clean and blackout accuracy of the fusion experiments (103 test clips, chance 0.20). 43

Table 5.10: Does the fusion model listen? Blackout: number of different classes predicted when all images are black. Audio swap: real images, audio shuffled between test clips. 44

Table 5.11: Fusion model under the hybrid and federated strategies. 45

Table 5.12: Latency per step for one clip. 47
Table 5.13: Update latency of the three strategies..... 47
Table 5.14: Predictions from the formulas compared with the measurements. 52
Table 6.1: Comparison with related work. 56
Table 6.2: My expectations from the exposé compared with the results. 57
Table 6.3: The five metrics from the exposé and how far they were measured. 58
Table 6.4: Differences between the exposé plan and the final work. 58
Table 7.1: Recommended strategy by situation..... 63
Table B.1: Strategy A, centralized..... 68
Table B.2: Strategy B, hybrid. 68
Table B.3: Strategy C, federated, local learning rate 1e-3. 68
Table B.4: Strategy C, federated, local learning rate 5e-4. 69
Table B.5: Strategy C, federated, local learning rate 1e-4. 69
Table B.6: Strategy C, federated, local learning rate 5e-5. 69
Table B.7: FB, hybrid fusion. 70
Table B.8: FC, federated fusion..... 70

List of Abbreviations

Abbreviation	Meaning
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
FB	Hybrid strategy with the fusion model
FC	Federated strategy with the fusion model
FedAvg	Federated Averaging
FL	Federated Learning
FV	Feature Vector
IID	Independent and Identically Distributed
LR	Learning Rate
MQTT	Message Queuing Telemetry Transport
QoS	Quality of Service
RMS	Root Mean Square
STFT	Short-Time Fourier Transform
YOLO	You Only Look Once (object detector)

1 Introduction

This chapter explains why I chose this topic, what problem I worked on, which questions I wanted to answer and how the rest of the thesis is organised.

1.1 Motivation

I chose this topic because robots, edge devices and machine learning are growing fast and already play an important role in many fields, from delivery and rescue robots to self-driving cars, and they will matter even more in the future.

Robots and autonomous systems mostly rely on cameras to recognise objects. Modern object detectors such as YOLO can name dozens of objects in a camera image within milliseconds [18], [19]. In fog, storms or darkness, however, the camera has difficulty detecting objects accurately, and this is often exactly the moment when good perception matters most.

In such situations, sound can take over. A car still sounds like a car in the dark, and a dog still barks in fog. A microphone is cheap, small and does not need light. People do the same every day: when we cannot see something, we often still know what it is because we hear it. The idea behind my thesis is to give a machine this backup sense, so that it can still recognise objects by their sound when the camera fails, and to combine sound and image when both are available.

The idea itself is not new. Chan et al. combined vision and hearing on a sound-localising robot [26], and other work has shown that sound and video can teach each other [14]–[16]. What is less studied is how such a system should keep learning once it runs on small edge devices, and where that learning should happen.

At the start of this work I saw this as a trade-off between three things that pull in different directions: privacy (a microphone that is always on also records conversations), power (training on a small battery-powered device is expensive) and accuracy (a central server with all the data usually learns best). My supervisor asked me not to assume from the start that federated learning is the answer, but to compare the options fairly. That is what this thesis does.

1.2 Problem Statement

To recognise sounds, a neural network needs thousands of labelled examples such as "this sound is a car". Labelling audio by hand is slow and expensive, and it is impossible for a device that keeps recording new data in the field.

I solve the labelling problem with the camera. While there is enough light, a pre-trained object detector (YOLOv8) looks at the camera image and names what it sees. The sound recorded at the same moment gets that name as its label. In this way the sense that works now (vision) teaches the sense that will be needed later (hearing). This is called cross-modal supervision. Once the audio model is trained, it can recognise objects from sound alone, also when the camera sees nothing.

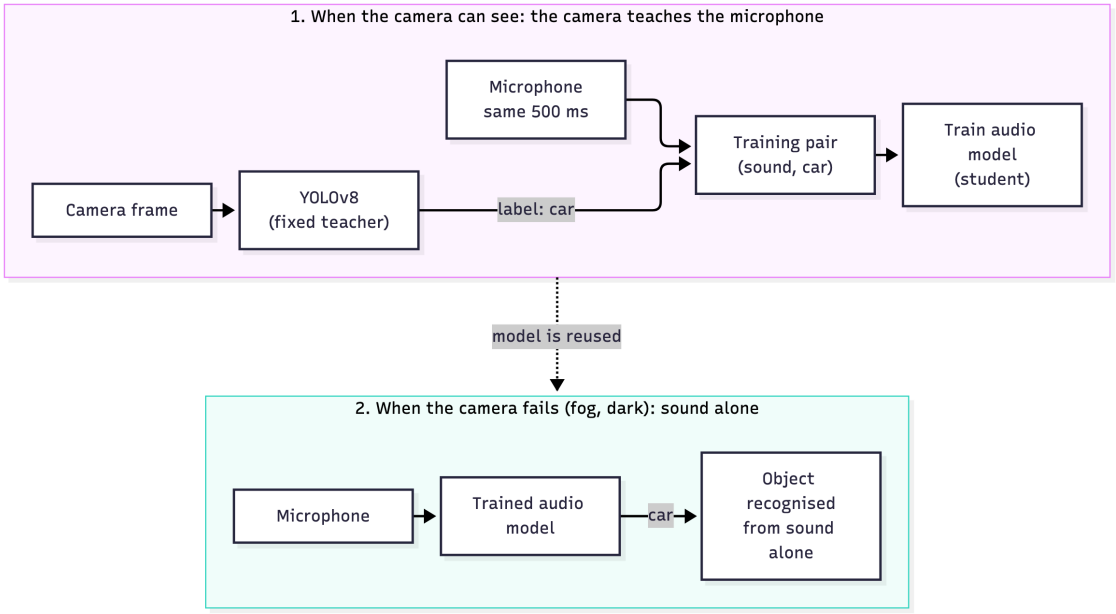


Figure 1.1: The core idea: while the camera can see, YOLOv8 labels the sound; when the camera fails, the trained audio model recognises the object from sound alone.
Source: own work.

Because the system labels its own data, it can keep learning for as long as it runs. This leads to the actual engineering question of my thesis: where should this training happen?

- on a central server, which receives the raw recordings from all devices (Strategy A, centralized),
- partly on the device, which prepares the data before sending it (Strategy B, hybrid), or
- completely on the device, which only shares what it has learned (Strategy C, federated).

Each choice changes how much data goes over the network, how much work the small device and the server must do, how private the recordings stay and how accurate the model becomes. These factors pull in different directions, and the right choice is not obvious without measuring.

It is important to keep two models apart from the start, because they do different jobs:

Table 1.1: The two models in the system and their roles.

	YOLOv8	**Audio / fusion model**
Role	teacher (gives the labels)	student (learns from the labels)
Trained in this thesis?	no, fixed and pre-trained	yes, this is the model I study
Input	camera image	audio (later: audio and image)
Needs light?	yes	no

1.3 Research Questions and Objectives

My exposé asked one main question: how do centralized, hybrid and federated training compare in accuracy, privacy, power consumption, bandwidth and convergence, and which one suits an audio-visual system on small edge devices? While working on it, I split this into three research questions:

RQ1. How do centralized, hybrid and federated training compare in accuracy, upload bandwidth, device workload, server workload and privacy?

RQ2. How does federated averaging (FedAvg) combine the model weights of the devices, and under which conditions does it reach the same accuracy as centralized training on edge devices?

RQ3. Can an audio-visual fusion model keep recognising objects when the camera fails, and does this ability survive hybrid and federated training?

To answer them, I set myself these objectives:

1. Build a pipeline that records sound at 16,000 samples per second, cuts it into 500 ms clips, turns each clip from the time domain into a mel spectrogram (frequency over time) with the short-time Fourier transform, and labels it automatically with YOLOv8.
2. Train a small audio classifier that is light enough for a Raspberry Pi.
3. Implement the three strategies so that they share the same devices, network protocol and data, and only the place of the work changes, including my own implementation of federated averaging, in which the server combines the devices' weight matrices with the FedAvg formula.

4. Measure accuracy, real network bytes and computing time for each strategy under the same conditions.
5. Build an audio-visual fusion model and test it with the camera blacked out.
6. Run the hybrid and federated strategies again with the fusion model.
7. Test the pipeline on a real Raspberry Pi with a live camera and microphone.

The original plan was to pick one strategy at the end. Instead, I implemented and measured all three, and in Chapter 7 I recommend which strategy fits which situation. Section 6.6 compares my results with the five metrics and my original predictions, including the ones I could not measure.

1.4 Contributions

What I am most proud of is that I did not only study these strategies on paper: I implemented them, built and tested the system, ran the experiments and observed what really happens, also on a real Raspberry Pi. The main contribution is therefore a measured, end-to-end comparison of where to train a self-labelling audio-visual edge system. Along the way, I found four further results:

1. **Preparing the data on the device saves little bandwidth.** Sending spectrograms instead of raw audio saved only 26% of the upload, because a half-second spectrogram is almost as large as the half second of audio it comes from. Federated learning needed 16.5 times less upload. Simple formulas explain all these numbers (Section 5.10).
2. **Federated averaging can damage a model that is already trained.** With the same learning rate as on the server, federated accuracy jumped up and down and fell below the starting point. A 20 times smaller local learning rate fixed this.
3. **A fusion model trained the obvious way ignores the weaker sense.** It learned to use only the camera. Warm-starting the audio branch and then using modality dropout made it use both senses and keep working with a black camera.
4. **Federated cost depends on model size, not on data volume.** The upload per federated round depends only on how many parameters the model has.

1.5 Structure of the Thesis

- **Chapter 2** explains the background: spectrograms, neural networks, YOLO, cross-modal supervision, federated learning, MQTT and multimodal fusion, together with related work.

- **Chapter 3** describes the system design, the three strategies, the datasets, the models and how I ran the experiments.
- **Chapter 4** describes the implementation on the laptop and on the Raspberry Pi, including the problems I ran into and how I solved them.
- **Chapter 5** presents the measured results and checks them against the formulas.
- **Chapter 6** discusses what the results mean, compares them with my original plan, and lists the limitations and my own reflections.
- **Chapter 7** answers the research questions, gives a recommendation and outlines future work.

2 Background and Related Work

This chapter explains the theory I needed for the rest of the thesis. I do not try to cover each field completely. For every formula I first say in plain words what it means, then give the formula itself and the book or paper it comes from. Each section ends with a short note on how the idea is used in my work.

2.1 Audio Signals and the Spectrogram

A microphone measures air pressure many times per second. After digitisation, a sound is just a long list of numbers, called samples. I use a sample rate of 16,000 samples per second, so half a second of sound is 8,000 numbers. This rate is common for environmental sound because it captures frequencies up to 8 kHz, which covers most of what is useful in engines, voices, animals and sirens.

A classifier cannot do much with the raw list of numbers, because the same sound can look very different depending on its loudness and phase. Most audio classifiers therefore first turn the sound into a spectrogram: a picture that shows how much energy there is at each frequency over time [11], [27].

Short-Time Fourier Transform (STFT). In words: cut the sound into short, overlapping pieces, fade the edges of each piece, and measure which frequencies each piece contains. Written as a formula [27], [34]:

$$X(m,k) = \sum_{n=0}^{N_{FFT}-1} x(n+mH)\, w(n) e^{-j2\pi kn/N_{FFT}}$$

where

- $x(n)$ is the audio signal,
- m is the number of the piece (frame) and k the frequency bin,
- H is the hop, the step from one frame to the next,
- $w(n)$ is the window that fades the edges of each frame,
- N_{FFT} is the FFT size,
- $X(m,k)$ is the result: how strong frequency k is in frame m.

Table 2.1 lists the settings I used and why.

Table 2.1: STFT settings.

Parameter	Value	Reason
Sample rate	16,000 Hz	covers 0–8 kHz, standard for environmental sound
Clip length	500 ms (8,000 samples)	short enough to react quickly, long enough to hold a sound event
Window length	25 ms (400 samples)	a sound hardly changes within this time
Hop length	10 ms (160 samples)	60% overlap between frames
FFT size	512	next power of two above 400, gives 257 frequency bins
Mel bands	128	groups the 257 bins into 128 bands that match human hearing

Number of frames. In words: the number of frames is how many times the window can jump by one hop along the clip, plus the first position. With the edge padding that librosa adds by default [12]:

$$T = 1 + \left\lfloor \frac{N}{H} \right\rfloor = 1 + \left\lfloor \frac{8000}{160} \right\rfloor = 51$$

where T is the number of frames, N the number of samples in the clip, H the hop in samples, and $\lfloor \cdot \rfloor$ means rounding down. Every 500 ms clip therefore becomes a picture of 51 time frames by 128 mel bands. Without the edge padding it would be $1 + \lfloor (8000 - 400)/160 \rfloor = 48$ frames.

Mel scale. In words: our ears hear small differences between low notes much better than between high notes, so low frequencies should get more detail. The standard mel formula [28] is

$$\text{mel}(f) = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

where f is a frequency in hertz and $\text{mel}(f)$ is the same frequency on the hearing scale. The constants are chosen so that 1,000 Hz equals 1,000 mel. The 257 frequency bins are collected into 128 triangular filters that are evenly spaced on the mel scale [27]. librosa, which I use, applies the closely related Slaney version of the mel scale by default: linear below 1 kHz and logarithmic above it. Both versions have the same shape and the same purpose [12].

Log scaling. In words: add up the energy inside each mel band and write it in decibels, because sound energy covers a huge range. As a formula [27]:

$$S(m, b) = 10 \cdot \log_{10}\left(\sum_k |X(m, k)|^2 M_b(k)\right)$$

where $S(m, b)$ is one pixel of the final picture (frame m, mel band b), $|X(m, k)|^2$ is the energy in frequency bin k, and $M_b(k)$ is how much bin k belongs to mel band b. Finally, I scale every spectrogram to the range 0 to 1.

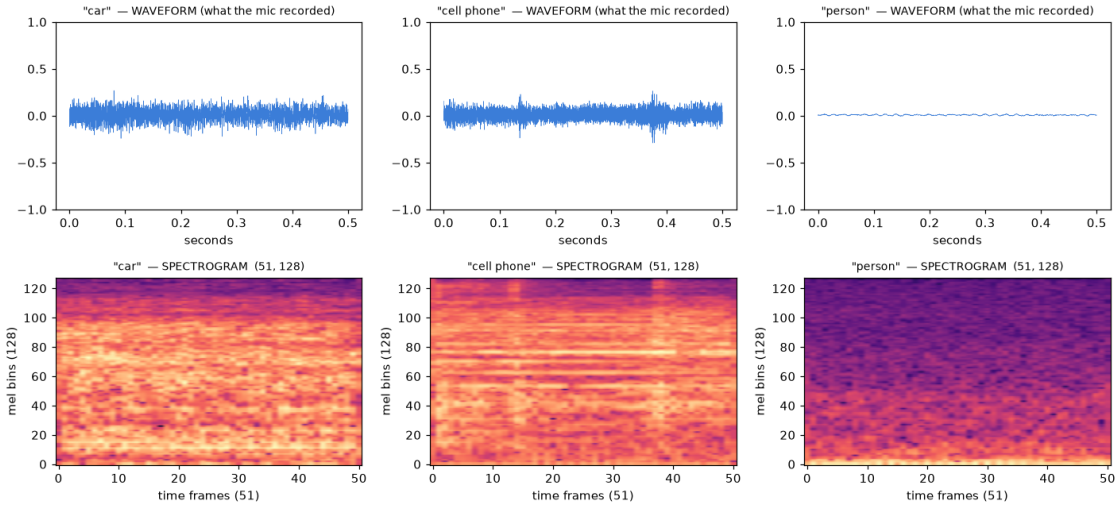


Figure 2.1: Three 500 ms clips recorded by the Pi (car, cell phone, person) as waveforms (top) and as log-mel spectrograms (bottom).
Source: own work.

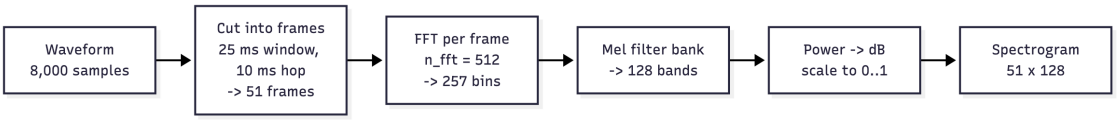


Figure 2.2: Steps from waveform to the 51 × 128 spectrogram.
Source: own work.

In my work: every 500 ms clip becomes a 51 × 128 picture, which the model treats like a small grey-scale image.

2.2 Convolutional Neural Networks for Sound

A convolutional neural network (CNN) learns small filters that slide over an image and react to local patterns, such as edges. Deeper layers combine these into bigger patterns [29]. Because a spectrogram is a picture of sound, the same idea works for audio: the first filters react to short tones or bursts of noise, later layers to the typical shape of a siren or a dog bark. Hershey et al. showed that normal image CNNs work well for large-scale audio classification [11].

Each block of my network has a convolution, batch normalisation [31], a ReLU activation and max pooling, which halves the size of the picture. At the end, a dense layer gives one score per class.

Softmax. In words: turn the scores into probabilities that are all positive and add up to 100%. The formula [29] is

$$p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}$$

where z_c is the raw score of class c, e^{z_c} makes it positive, and dividing by the sum over all classes j makes the probabilities p_c add up to 1.

Cross-entropy loss. In words: the loss is small when the model gives a high probability to the right answer and large when it is confident and wrong. The formula [29] is

$$\ell = -\sum_c y_c \log p_c$$

where y_c is 1 for the correct class and 0 for all others. Only the term for the correct class is left, so the loss is simply $-\log$ of the probability the model gave to the right answer.

Gradient descent. In words: move every weight a small step in the direction that makes the loss smaller. The basic rule [29] is

$$w \leftarrow w - \eta \nabla_w \ell$$

where w stands for all the weights of the network, $\nabla_w \ell$ (the gradient) tells for each weight in which direction the loss would grow, the minus sign makes the step go the other way, and η is the learning rate, the size of the step. I use the Adam optimiser [30], which adapts the step size for each weight but follows the same idea. The learning rate turned out to be the most important setting for federated learning (Section 5.4).

In my work: the audio classifier is a small CNN with three convolution blocks and 111,370 parameters, small enough to train on a Raspberry Pi.

2.3 Object Detection as a Teacher: YOLO

YOLO (You Only Look Once) is a family of object detectors that find objects and name them in one pass over the image [18]. YOLOv8 [19] knows the 80 everyday classes of the COCO dataset, such as person, car, dog, bird and cell phone, and gives every detection a confidence between 0 and 1.

I do not train or change YOLO. It is only my labelling tool. If YOLO finds an object with a confidence of at least 0.5, the audio recorded at the same moment gets that name. If YOLO is not sure, the clip gets no label and is skipped.

In my work: I use the smallest version, YOLOv8n. It needs about 36 ms per frame on the laptop and about 318 ms per frame on the Raspberry Pi 5, which is fast enough for one label every 500 ms.

2.4 Cross-Modal and Weak Supervision

Cross-modal supervision means that one sensor provides the labels for another. Aytar et al. used this idea in SoundNet: they trained a sound network to copy what image networks saw in millions of unlabelled videos [14]. Owens et al. used sound to teach a vision model [15], and Arandjelović and Zisserman learned audio and image features together simply from the fact that they happen at the same time [16]. Combining a camera and a microphone on a robot was also shown by Chan et al. [26].

Labels made this way are noisy. The camera may see a person while the sound comes from a car outside the picture. This is called weak supervision. Deep networks can live with a surprising amount of label noise, because wrong labels are often random and disagree with each other, while correct labels keep pointing to the same pattern [17].

In my work: I measured the noise instead of assuming it. On VGGSound images, YOLO's label matched the human sound label in 85% of the cases where it detected something, so about 15% of the automatic labels are wrong (Section 5.1.1).

2.5 Federated Learning

2.5.1 The idea

In normal (centralized) machine learning, all data is collected on one server and the model is trained there. In federated learning (FL), the data stays on the devices. Each device trains its own copy of the model on its own data and sends only the updated model back. The server combines these models into a new shared model and sends it out again [1], [2]. A short way to say it: the data stays home and the model travels.

This has three advantages for edge systems:

- **privacy:** raw recordings never leave the device,
- **bandwidth:** a model can be much smaller than the data,
- **server load:** the server only averages, it does not train.

The price is that every device must be able to train a model.

2.5.2 The FedAvg algorithm

FedAvg was introduced by McMahan et al. [1] and is still the standard algorithm. One round works like this:

1. The server sends the current shared model w_t to all clients.
2. Each client copies it and trains for E epochs on its own data, using the gradient descent rule from Section 2.2 with learning rate η. The result is that client's local model w_{t+1}^k.
3. Each client sends its model and the number of samples n_k it trained on back to the server.
4. The server averages the models, giving more weight to clients with more data.
5. The new shared model goes out, and the next round starts.

The aggregation step. In words: the new shared model is a weighted average of the clients' models, and each client's weight is its share of all the data. The formula from [1] is

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k$$

where K is the number of clients, n_k the number of samples on client k, $n = \sum_k n_k$ the total number of samples, and n_k/n the share of client k. With my two edge nodes, A and B, this becomes

$$w_{t+1} = \frac{n_A}{n_A + n_B} w_A + \frac{n_B}{n_A + n_B} w_B$$

A small example for one single weight: if node A trained on 3,000 samples and ended with the value $w_A = 0.40$, and node B trained on 1,000 samples and ended with $w_B = 0.80$, then

$$w_{t+1} = \frac{3000}{4000} \cdot 0.40 + \frac{1000}{4000} \cdot 0.80 = 0.30 + 0.20 = 0.50$$

The node with more data pulls the result towards its own value.

What the clients are trying to achieve together. In words: the goal is the same as in normal training, a small loss over all data of all clients, but each client only computes its own part [1]:

$$F(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w)$$

where $F(w)$ is the loss over all data and $F_k(w)$ is the average loss on client k's own data. The server never needs the data to work towards this goal, because each client reports only its own part.

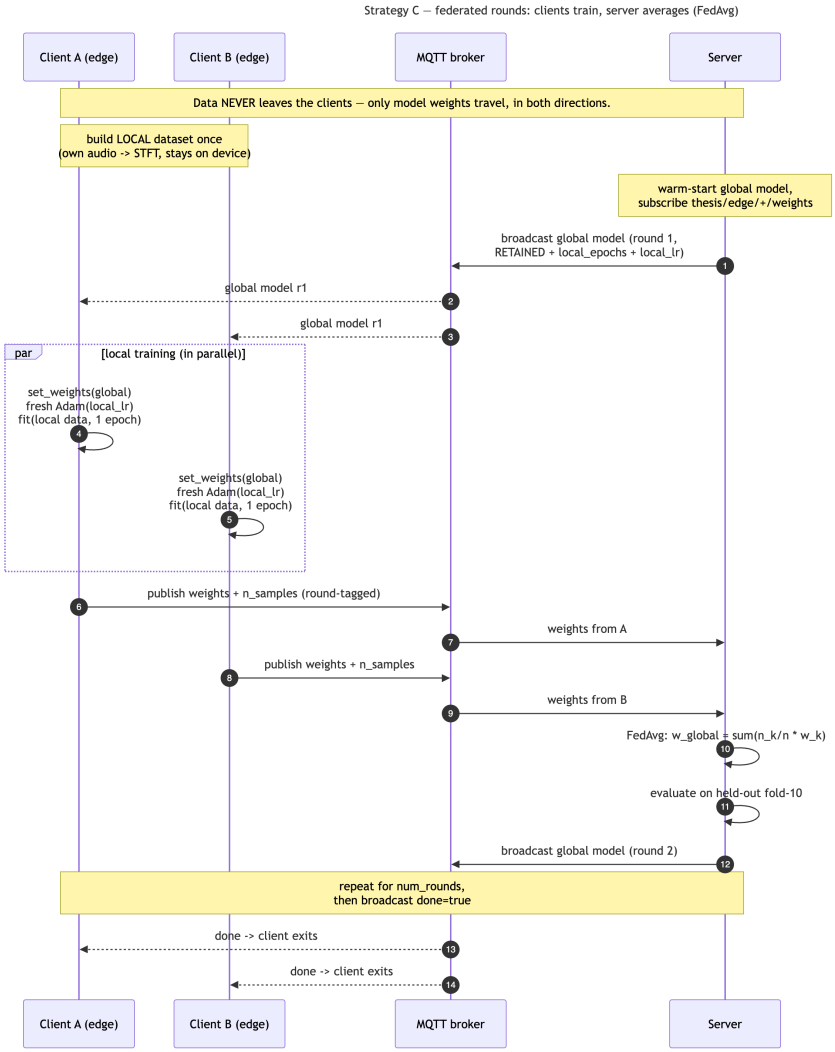


Figure 2.3: One FedAvg round between the server and two edge nodes.
Source: own work.

2.5.3 What actually happens to the weight matrices

The step I found hardest to picture is what "averaging two models" means in practice. In a discussion, my supervisor asked a good question: if two Raspberry Pis learn different things, would their weights not cancel each other out when the rows and columns of their matrices are averaged? To answer it, I went back to the original FedAvg paper [1]. The answer is in the next two sections. A model is not one number but a list of matrices. My AudioCNN, for example, is a list of 22 arrays with 111,370 numbers in total (Table 2.2).

Table 2.2: The weight arrays of the AudioCNN.

Layer	Array	Shape	Numbers
Conv block 1	filter weights	3 × 3 × 1 × 32	288
Conv block 2	filter weights	3 × 3 × 32 × 64	18,432
Conv block 3	filter weights	3 × 3 × 64 × 128	73,728
Dense layer	weight matrix	128 × 128	16,384
Output layer	weight matrix	128 × 10	1,280
all layers	biases and batch-norm values	various	1,258
total	**22 arrays**		**111,370**

Source: read from the trained model with Keras.

FedAvg treats each of these arrays separately and averages them element by element: the number in row 1, column 1 of node A's matrix is averaged with the number in row 1, column 1 of node B's matrix, and so on. For a tiny 2 × 2 matrix with equal data on both nodes:

$$\frac{1}{2}\begin{pmatrix}0.2 & -0.4\\ 0.6 & 0.1\end{pmatrix}+\frac{1}{2}\begin{pmatrix}0.4 & -0.2\\ 0.2 & 0.3\end{pmatrix}=\begin{pmatrix}0.3 & -0.3\\ 0.4 & 0.2\end{pmatrix}$$

The server does exactly this for all 22 arrays, which in my code is one loop over the layers (Listing 4.1). The shapes never change; only the values inside do.

2.5.4 Why averaging weights works at all

Averaging two networks only makes sense if they started from the same weights. A network can solve the same task in many equivalent ways: swapping two neurons in a layer, together with their connections, gives the same result but different weight values. If the two clients started from different random weights, neuron 5 in one model might do the job of neuron 12 in the other, and the element-by-element average from Section 2.5.3 would mix unrelated neurons. Because every FedAvg round starts all clients from the same shared model w_t, the neurons stay aligned and the average stays meaningful [1]. It is a design requirement of FedAvg, not an assumption.

2.5.5 Local steps and client drift

The more steps a client takes on its own, the further its weights move away from the shared model. Roughly, this distance grows with the learning rate η, the number of local epochs E and the number of batches. If the clients' data are different (non-IID data), each client moves in its own direction and the average can end up in a poor place. This is called client drift [2], [3]. FedProx reduces it by punishing a client for moving too far from the shared model [4]:

$$\min_w F_k(w) + \frac{\mu}{2}\|w - w_t\|^2$$

where $\|w - w_t\|^2$ measures how far the local model has moved from the shared model w_t, and μ sets how strongly this is punished. I did not use FedProx, but the same effect, local models moving too far, appeared in my experiments even with evenly split data (Section 5.4).

2.5.6 Communication cost

In words: in federated learning each client uploads its model, so the upload per round is the number of model parameters times the size of one number. Since every parameter is a 32-bit float (4 bytes), the upload follows directly from the model size (my own derivation, based on the setting in [1]):

$$B_{\text{round}} \approx P \times 4 \text{ bytes}$$

where B_{round} is the upload per client and round and P is the number of parameters sent. For my audio model with 111,370 parameters this is about 445 KB per round, no matter how many samples the client trained on. In centralized training the upload grows with every new sample instead. This simple formula explains one of the main results in Chapter 5.

2.5.7 Privacy limits

Federated learning keeps the raw data on the device, but it is not perfectly private. Zhu et al. showed that training data can sometimes be rebuilt from shared gradients [7]. Secure aggregation [8] and differential privacy are the usual ways to reduce this risk, and they are outside the scope of my thesis. My privacy claim is therefore limited: raw audio never leaves the device, not that no information at all leaves it.

2.5.8 Frameworks

Flower [5] is a popular framework for federated learning, and Ridolfi et al. used it on Raspberry Pi devices [6]. Flower was my first choice, and an early version of my project did. The final system implements FedAvg directly over MQTT, for the reason given in Section 3.3.

2.6 Edge Computing and MQTT

Edge computing means processing data close to where it is created, on small devices such as a Raspberry Pi, instead of sending everything to a data centre. This saves network traffic and delay, but edge devices have little memory, computing power and energy.

MQTT is a lightweight publish/subscribe protocol made for such devices [24]. Devices do not talk to each other directly. A central broker (in my case Eclipse Mosquitto [25]) receives every message. A sender publishes a message to a topic, a text label that looks like a file path, and every client that has subscribed to that topic receives it. I use three MQTT features:

- **Quality of Service (QoS) 1:** the broker makes sure a message arrives at least once, so a batch cannot get lost without anyone noticing.
- **Retained messages:** the broker keeps the last message on a topic and gives it to every client that subscribes later. I use this for the shared model, so a device that connects late still gets the current model.
- **Wildcards:** the server subscribes to `thesis/edge/+/data`, where `+` matches any node name, so one subscription receives data from all devices.

2.7 Multimodal Fusion and Modality Imbalance

Multimodal models combine different kinds of input, for example an image and a sound. In early fusion the raw inputs are joined before any processing. In late fusion each input first goes through its own branch, and the results are joined near the end. Late fusion is simpler and lets me reuse pre-trained branches, so I use it.

A known problem is that multimodal networks are often no better than their best single input. Wang et al. showed that the input that learns fastest takes over the training, and the others stay under-trained [21]. Peng et al. call this modality imbalance and balance the learning of the branches [22]. A simpler countermeasure is modality dropout (ModDrop), introduced by Neverova et al. [23]: during training, one input is randomly removed so the model cannot depend on it alone. It builds on the general dropout idea of Srivastava et al. [32].

For the vision branch I use MobileNetV3-Small [20], a small image network made for mobile devices and pre-trained on ImageNet.

In my work: exactly the imbalance described in [21] appeared. Section 5.7 shows how I found it and fixed it.

2.8 Research Gap

Many federated learning studies compare final accuracy on benchmark datasets in simulation. Fewer measure the actual bytes that cross a real network when the same work is placed in different locations, and I found none that do this for a system that labels its own data with a second sensor. Work on audio-visual learning, on the other hand, usually assumes that all data is on one machine. My thesis connects the two: it measures centralized, hybrid and federated placement for a self-labelling audio-visual system on real messaging infrastructure, and it checks whether a fusion model's ability to work without the camera survives federated training.

3 System Design and Methodology

This chapter describes what I built and how I measured it, in enough detail that someone else could repeat my experiments. Chapter 4 then explains how I implemented the design.

3.1 Research Approach

My thesis is a comparative experimental study. I carry out the same task, learning to recognise objects from sound, with three different placements of the training, and I measure each placement with the same metrics on the same data.

To keep the comparison clean, I worked in two stages and changed only one thing at a time:

1. **Stage one, audio only.** The three strategies were first compared with a small audio-only model. This isolates the effect of the placement from the complexity of the model.
2. **Stage two, audio and vision.** A fusion model that uses both the camera image and the audio was then built, tested with the camera blacked out, and run again under the hybrid and federated strategies.

Finally, I ran the pipeline on a real Raspberry Pi to show that the results are not just an effect of the simulation.

3.2 System Architecture

The system has three kinds of machines (Figure 3.1).

Edge nodes. Raspberry Pi computers with a camera and a microphone. They sit where the data is produced, they are weak in compute and they are often on a slow or expensive network. The comparison experiments use two edge nodes, called A and B, which run as two separate processes on the laptop (Section 3.7); the live tests use one Raspberry Pi 5.

Server. A laptop with no sensors. It is powerful, it holds the test set and it keeps the official global model that all nodes share.

Broker. A Mosquitto MQTT broker, which runs on the same laptop. Edge nodes and server never talk to each other directly; all messages go through the broker.

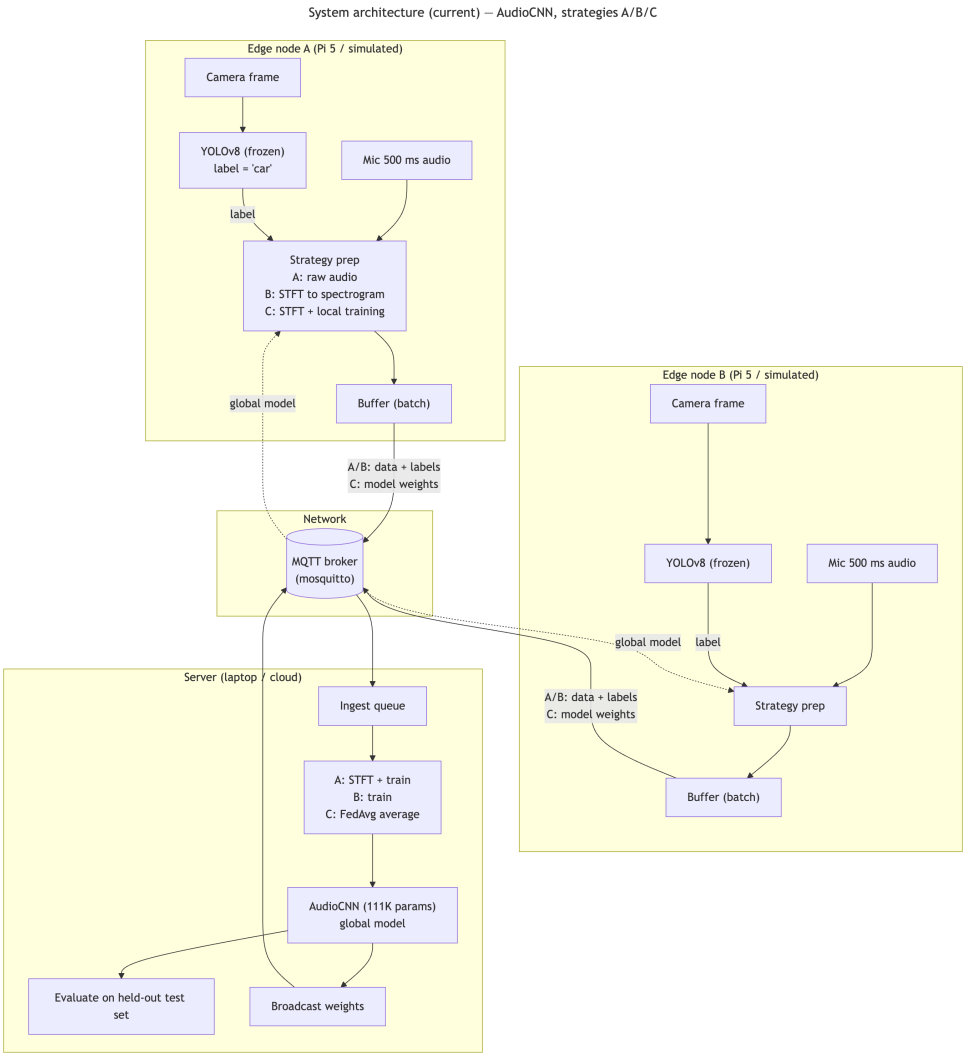


Figure 3.1: System architecture with two edge nodes, the MQTT broker and the server.
Source: own work.

3.2.1 The edge loop

Each edge node repeats the same loop every 500 ms. One pass of the loop is called a tick:

1. Record 500 ms of audio from the microphone.
2. Take one camera frame at the same moment.
3. Run YOLOv8 on the frame. If an object is detected with a confidence of at least 0.5, its class becomes the label. If not, the tick is skipped.
4. Discard the image. It is never sent, and it is only stored in an optional evidence mode that I used to build the demo gallery (Section 5.6).
5. Store the pair {audio, label} in a local buffer.

Nothing is sent during a tick. Training in this system is batched, not real-time.

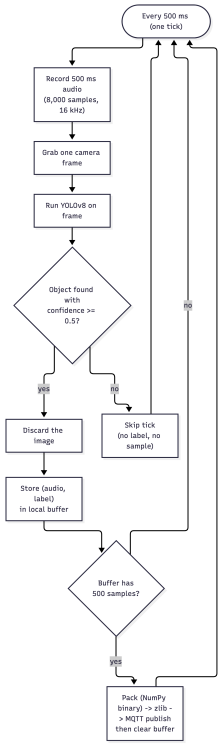


Figure 3.2: The edge loop that runs every 500 ms.
Source: own work.

3.2.2 Batching

Data is sent in batches of 500 samples. Sending each sample on its own would add the MQTT and TCP headers 500 times, keep the radio active all the time and compress badly. One message with 500 samples carries the overhead once and compresses much better.

When the buffer is full the node:

1. stacks all audio clips into one array and stores it in NumPy's binary format, with the labels and timestamps in a small JSON block,
2. compresses the result with zlib, and
3. publishes it as a single MQTT message.

Binary storage matters: a 16-bit sample takes 2 bytes in binary, but around 6–10 bytes when written as text in JSON.

3.3 Communication Design

All communication uses MQTT with the topics in Table 3.1.

Table 3.1: MQTT topics.

Topic	Direction	Content
`thesis/edge/{id}/data`	edge → server	Strategy A/B: a batch of audio or spectrograms
`thesis/edge/{id}/weights`	edge → server	Strategy C: locally trained weights
`thesis/server/model/global`	server → edges	the current global model

All messages use QoS 1. The global model in the federated strategy is published as a retained message, so a node that connects late receives it immediately. In the federated strategies every message carries a round number, and the server ignores messages from rounds that are already closed.

On the server, the MQTT callback only puts incoming messages into a queue and returns. A separate loop takes messages from the queue, decodes them and starts training. Training inside the network callback would block the connection for many seconds and could lead to lost messages.

Why not Flower. The Flower framework [5] was planned at first for federated learning, and an early version of the project did. In the final system FedAvg was implemented directly on top of MQTT. The reason is measurement fairness: if Strategy C used Flower's gRPC transport while A and B used MQTT, the bandwidth numbers would compare two different protocols, not two placements. With one transport for all strategies, every byte is counted in the same way.

3.4 The Three Strategies

The machines, the network and the data are the same in all three strategies. Only the division of work changes (Table 3.2).

Table 3.2: Division of work in the three strategies.

	Edge does	Network carries	Server does
A, centralized	record, label	raw audio	STFT, train, evaluate
B, hybrid	record, label, STFT	spectrograms	train, evaluate
C, federated	record, label, STFT, train	model weights only	average (FedAvg), evaluate

In all three strategies, the trained model makes its decisions on the device. In a fully centralized variant, the device could also send every sound to the server and receive the decision back (cloud inference). In this work, the server sends the trained model back and the device makes the decision itself, so it keeps working without a network connection, as in fog or darkness.

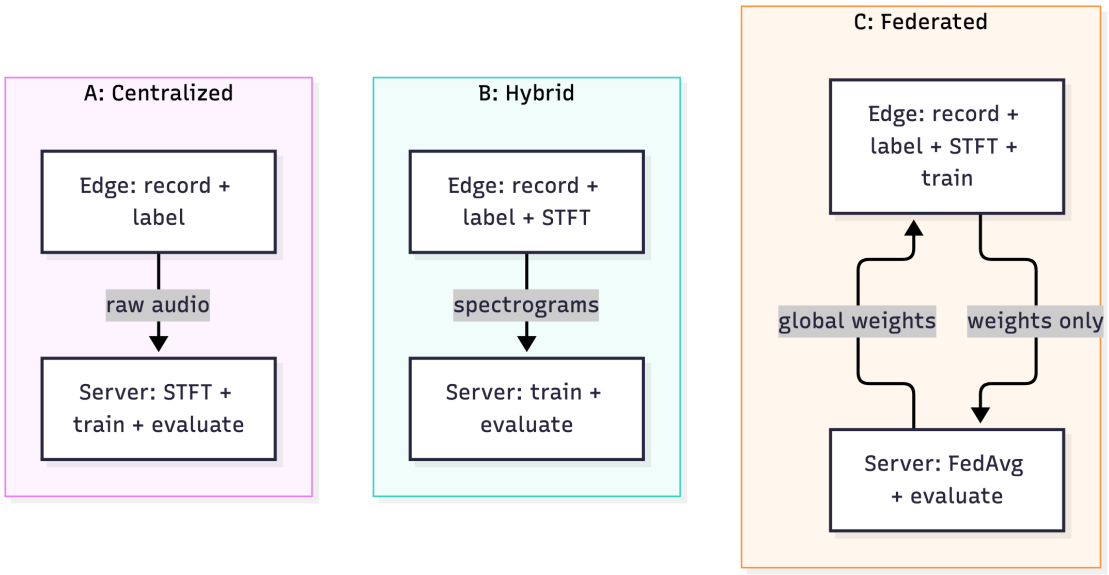


Figure 3.3: What the edge does, what crosses the network and what the server does in each strategy.
Source: own work.

From A to C, one job at a time moves from the server to the device. The expected effect is less data on the network and more privacy, paid for with more work on the device.

Strategy A, centralized. The device only records, labels and sends raw 16-bit audio. The server converts the audio to spectrograms, adds them to a growing pool and, every time 1,000 new samples have arrived, trains the model for one epoch on everything it has received so far. It then evaluates the model and broadcasts it.

Strategy B, hybrid. Exactly one thing changes: the STFT moves to the device. The device sends spectrograms as 16-bit floats instead of raw audio. Everything else is identical to A.

Strategy C, federated. The server no longer trains. In each round it sends the global weights; every node trains one local epoch on its own data and sends back its weights together with its sample count; the server averages them with FedAvg (Section 2.5.2), evaluates the result and broadcasts it. The server waits until both nodes have reported before it averages.

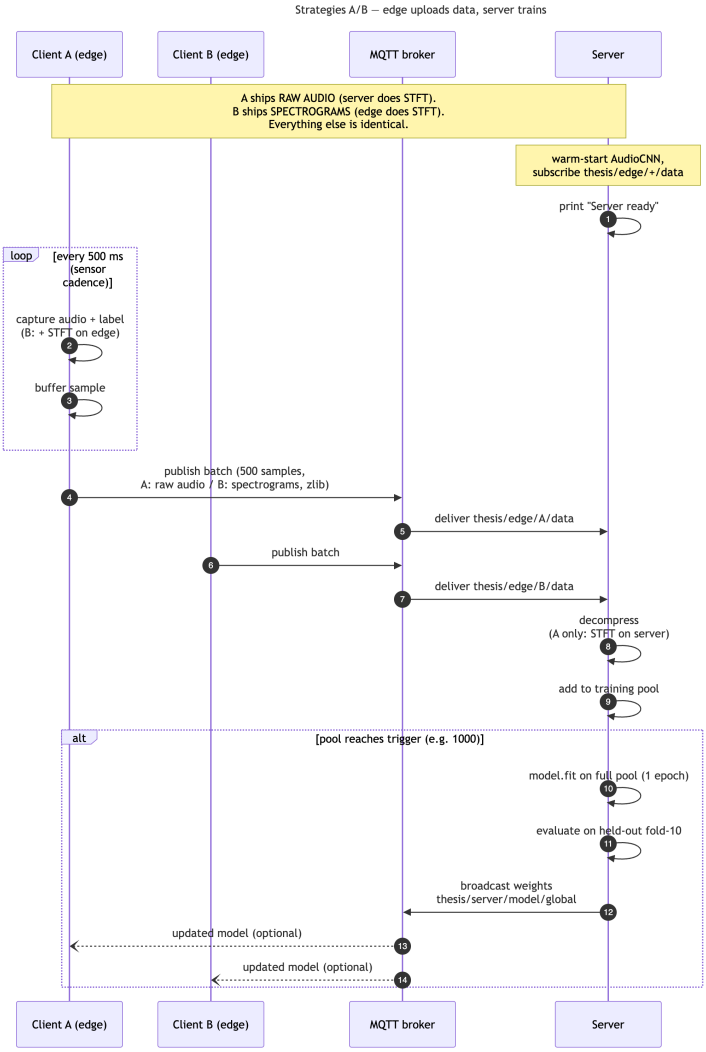


Figure 3.4: Message sequence for strategies A and B.
Source: own work.

3.5 Datasets

3.5.1 UrbanSound8K

UrbanSound8K [9], a standard benchmark for environmental sound classification like ESC-50 [10], contains 8,732 labelled sound clips of up to 4 seconds in ten urban classes: air conditioner, car horn, children playing, dog bark, drilling, engine idling, gun shot, jackhammer, siren and street music. It is split into ten official folds. In this work folds 1–8 are used for training, fold 9 for validation and fold 10 only for the final test, so every reported accuracy is measured on recordings the model has never seen.

A public dataset was needed because accuracy can only be measured if the correct answer is known. Recordings made live with the Raspberry Pi have no reliable ground truth.

3.5.2 VGGSound

UrbanSound8K has no video, so it cannot be used to train a model that also looks at images. VGGSound [13] contains about 200,000 ten-second YouTube clips with human labels for the sound. The authors only publish the video IDs and timestamps, so the clips must be downloaded by each user.

Five classes were chosen that YOLO can see and that also make a typical sound: car, motorcycle, dog, cat and bird. For each clip, the middle frame and the 16 kHz audio were extracted. The resulting dataset has 987 clips, between 195 and 200 per class.

The split into training, validation and test sets (80/10/10) is done **by clip**, not by audio window. Several 500 ms windows are cut from each ten-second clip, and windows from the same clip are almost identical. If some windows of a clip were in the training set and others in the test set, the model would be tested on near copies of its training data. This is called data leakage and it would make the results look better than they are.

The labels used for the fusion experiments are the human labels from VGGSound, not YOLO labels, so that the accuracy can be measured correctly. The quality of YOLO as a labeller was measured separately (Section 5.1).

3.6 Models

3.6.1 AudioCNN

The audio classifier takes a 51 × 128 × 1 spectrogram as input (Table 3.3).

Table 3.3: AudioCNN architecture (111,370 parameters for 10 classes).

Layer	Output	Details
Input	51 × 128 × 1	log-mel spectrogram
Conv block 1	25 × 64 × 32	Conv 3×3 (32), BatchNorm, ReLU, MaxPool 2×2, Dropout 0.15
Conv block 2	12 × 32 × 64	Conv 3×3 (64), BatchNorm, ReLU, MaxPool 2×2, Dropout 0.15
Conv block 3	6 × 16 × 128	Conv 3×3 (128), BatchNorm, ReLU, MaxPool 2×2, Dropout 0.15
Global average pooling	128	
Dense	128	ReLU, Dropout 0.3
Dense (output)	N classes	softmax

The model was kept small on purpose, because in Strategy C it has to be trained on a Raspberry Pi.

3.6.2 Fusion model

The fusion model has two branches and a fusion head (Figure 3.5).

- **Vision branch.** MobileNetV3-Small [20], pre-trained on ImageNet and **frozen**, turns a 224 × 224 image into a 576-number feature vector. A small trainable dense layer projects it to 256 numbers.
- **Audio branch.** The same three convolution blocks as the AudioCNN, followed by a dense layer with 128 outputs. This branch is **trained**, because no pre-trained network knows what a dog sounds like in this setting.
- **Fusion head.** The 256 + 128 = 384 numbers are joined and passed through dense layers of 256 and 128 units with dropout 0.3, and a softmax output.

The model has about 1.33 million parameters, of which about 389,000 are trainable. This is called late fusion.

The vision branch is frozen for three reasons: a Raspberry Pi cannot afford to train it, its pre-trained features are already good for these everyday classes, and freezing it means its output never changes, which allows the device to send the feature vector instead of the image (Section 3.6.3).

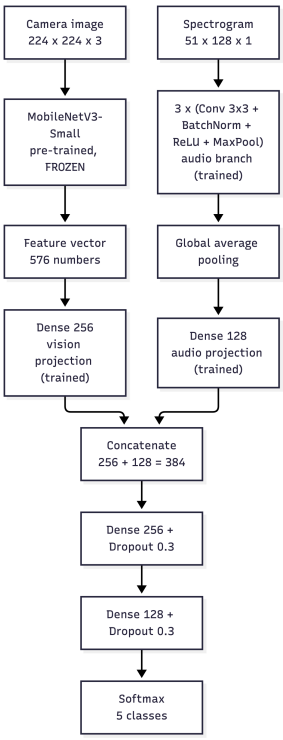


Figure 3.5: Fusion model architecture.
Source: own work.

3.6.3 Placing the fusion model in the strategies

When the fusion model is used in the strategies, one rule decides where each part lives:

> Frozen parts can stay on the device. Trainable parts must be wherever training happens.

A second rule is that camera images never leave the device. In the hybrid fusion strategy (FB), the server trains, so all trainable parts are on the server. The device runs the frozen MobileNetV3 and the STFT and sends the spectrogram together with the 576-number feature vector. The feature vector is about 1 KB, much smaller than a JPEG frame (on average 24 KB for the VGGSound frames and 66 KB for the Pi camera frames in my data), and it cannot easily be turned back into a recognisable picture. In the federated fusion strategy (FC), each device holds a full copy of all trainable parts and only weights are sent.

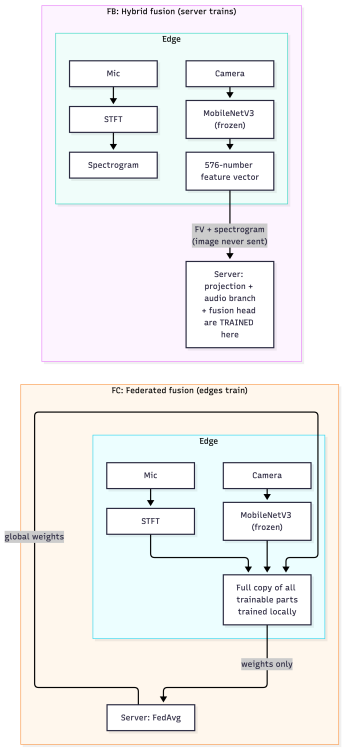


Figure 3.6: Placement of the fusion model in the hybrid (FB) and federated (FC) strategies.
Source: own work.

3.7 Experimental Protocol

All strategy experiments in stage one used the same setup:

- **Two edge nodes**, each a separate process with its own MQTT connection.

- **Disjoint data.** The training clips are shuffled with a fixed seed; node A then takes every even position and node B every odd position, so they never see the same sample. The class distribution on both nodes is almost the same (IID).
- **10 rounds.** In A and B each node streams **5,000 samples**. Node A owns 3,540 and node B 3,539 unique training clips (half of folds 1–8 each), so after the first 3,540 the clips are sent a second time, as a device in the field would keep hearing similar sounds. In C each node trains on all its own clips in every round.
- **Same starting point.** All runs start from the same trained AudioCNN (warm start), so the strategies are compared on equal terms.
- **Same evaluation.** After every round the server evaluates the model on fold 10.
- **Bytes are counted twice**, once by the sender and once by the receiver, and the two counts are compared.

3.7.1 Why the sensors are replayed

In the stage-one experiments the sensors are replayed: instead of recording from a live microphone, each node reads audio clips from UrbanSound8K and passes them on as if they had just been recorded, and the label comes from the dataset instead of from YOLO. Everything after that point is real: two separate client processes, a real MQTT broker, real serialisation and compression, real byte counts, real training and real evaluation.

I had three reasons for this choice.

1. **A fair comparison needs identical input.** If each strategy captured its own live audio, each would process different sounds, and any difference in accuracy or bandwidth could come from the data and not from the strategy. With replay, all strategies see exactly the same data with the same labels, so any difference is caused by the strategy. This is a controlled experiment.
2. **Accuracy needs ground truth.** To report that a model is 65% accurate you have to know the correct answers. UrbanSound8K provides them; a live recording in a room does not.
3. **The question does not depend on the sensor.** The thesis asks where training should run. That depends on what is computed and what is sent, not on whether the audio came from a microphone or from a file. The bytes are the same either way.

To make sure the replay does not hide a problem, I also ran the same pipeline live on a Raspberry Pi (Section 5.6).

3.8 Metrics

Table 3.4: Metrics used in the experiments.

Metric	Definition
Test accuracy	correct predictions on the test set divided by test set size
Upload per node	total bytes sent from one edge node to the broker, counted at sender and receiver
Device compute	seconds spent on the device for STFT and/or training
Server compute	seconds per round spent on the server for training or averaging
Data leaving the device	what kind of data crosses the network (raw audio, spectrograms, weights)
Blackout accuracy	test accuracy when every image is replaced by a black image (fusion only)

The blackout test is the main test for the claim that audio helps when vision fails. Every fusion model is evaluated twice on the same test clips: once with the real images and once with all images replaced by black images, while the audio stays the same. This simulates complete loss of vision, as in darkness or with a broken camera.

3.9 Tools and Use of AI Assistance

Software and tools. All code was written in Python. Models were built and trained with TensorFlow/Keras, audio was processed with librosa and NumPy, objects were detected with Ultralytics YOLOv8, and messages were sent with paho-mqtt through a Mosquitto broker. The result charts in Chapter 5 were made with Matplotlib directly from the JSON result files that each experiment writes, so every chart shows exactly the stored numbers. The architecture and flow diagrams were drawn with Mermaid, a tool that turns a short text description into a diagram; their sources are kept in the repository next to the images. No figure was taken from another publication.

AI assistance. I designed the project and the experiments, built and tested the hardware, and used an AI assistant as a tool for programming, running experiments, making the charts and drafting the text. I checked every result, and I rewrote key parts in my own words.

4 Implementation

This chapter explains how I turned the design from Chapter 3 into working software and hardware. I focus on the decisions that mattered and on the problems I ran into, not on listing code. The full source code is in my project repository (Appendix D).

4.1 Hardware and Software

Table 4.1: Hardware.

Component	Used in this work
Edge device	Raspberry Pi 5 Model B, 8 GB RAM, Debian 13
Camera	USB webcam, 1080p, MJPG
Microphone	separate USB microphone
Server	MacBook Air laptop (Apple M1, 8 cores, 8 GB RAM, 256 GB SSD)
Network	Pi and laptop on the same local network

Table 4.2: Software.

Software	Version	Used for
Python	3.11 (laptop), 3.13 (Pi)	everything
TensorFlow / Keras	2.21	AudioCNN, fusion model, training
librosa	0.11	STFT and mel spectrogram
Ultralytics YOLOv8	8.4	object detection (labelling)
paho-mqtt	2.1	MQTT client
Eclipse Mosquitto	2.1.2	MQTT broker
NumPy	2.4	arrays, serialisation
yt-dlp, ffmpeg	–	VGGSound download and frame/audio extraction

The code is organised in three parts: `src/edge` for code that runs on the device (sensors, STFT, YOLO wrapper, MQTT client), `src/server` for the models and the FedAvg aggregator, and `src/experiments` for the clients and servers of each strategy. The script `scripts/run_strategy.py` starts one server and two clients for a chosen strategy and writes all results to a timestamped folder.

4.2 Audio Pipeline

The STFT is implemented in `src/edge/processing/stft.py` with librosa using the parameters in Table 2.1. The steps for each 500 ms clip are:

1. compute the mel power spectrogram (n_fft 512, window 400, hop 160, 128 mel bands, 0–8 kHz),
2. convert power to decibels relative to the maximum,

3. scale the result to the range [0, 1].

The output has the shape 51 × 128. The same code runs on the laptop (A) and on the Raspberry Pi (B, C), so both sides produce exactly the same spectrograms.

4.3 Labelling Pipeline

YOLOv8n is loaded once at start-up. For each tick it runs on the current frame; the detection with the highest confidence above 0.5 gives the label. Only a list of classes of interest is kept, and during live enrollment further classes can be excluded with an ignore list. The frame is then dropped. On the Raspberry Pi 5 CPU, YOLO took about 318 ms per frame, which fits within the 500 ms tick.

4.4 Client and Server Logic

The logic of the clients and servers is shown as pseudocode. Strategies A and B share the same structure; only the payload differs.

Edge client, strategies A and B

```
loop every 500 ms:
    audio  = record_500ms()
    label  = yolo(frame)            # skip tick if no confident detection
    if strategy == B: audio = stft(audio)
    buffer.append(audio, label)
    if len(buffer) == 500:
        payload = zlib(savez(buffer))
        mqtt.publish("thesis/edge/<id>/data", payload, qos=1)
        buffer.clear()
```

Server, strategies A and B

```
on message: queue.put(message)        # return immediately
loop:
    batch = decode(queue.get())
    if strategy == A: batch = stft(batch)
    pool.extend(batch)
    if new samples since last round >= 1000:
        model.fit(pool, epochs=1)       # train on everything received
        evaluate(model, fold10)
        mqtt.publish("thesis/server/model/global", model.weights)
```

Federated round, strategy C

```
server: publish global weights (retained), round = r
client: model.set_weights(global)
        model.fit(local_data, epochs=1, lr=local_lr)   # fresh Adam each round
        publish("thesis/edge/<id>/weights", weights, n_samples, round=r)
server: wait until both clients reported for round r
        w = sum(n_k / n * w_k)          # FedAvg, layer by layer
        evaluate(w, fold10); r = r + 1
```

FedAvg is implemented in `src/server/aggregation/fedavg.py`. The core of the aggregation is only a few lines and follows the formula in Section 2.5.2 directly (Listing 4.1).

```
total_samples = sum(sample_counts)
aggregated = []
for layer_idx in range(n_layers):
    layer_weights = [client_weights[i][layer_idx] for i in range(n_clients)]
    weighted_sum = np.zeros_like(layer_weights[0])
    for i, weights in enumerate(layer_weights):
        weighted_sum += (sample_counts[i] / total_samples) * weights
    aggregated.append(weighted_sum)
```

Listing 4.1: Weighted averaging of client weights (FedAvg), shortened from `src/server/aggregation/fedavg.py`.

The aggregator averages every weight array of the model, including the batch-normalisation statistics, weighted by the number of samples each client used. The optimiser state is not shared; each client starts a fresh Adam optimiser every round, so that no optimiser state is shared; the original FedAvg algorithm uses plain SGD on the clients [1].

Serialisation. Strategy A sends 16-bit integer audio, Strategy B 16-bit float spectrograms, and Strategy C 32-bit float weights. In all cases the arrays are packed with NumPy's compressed format together with a small JSON header. In strategies A and B the result is compressed once more with zlib before publishing.

4.5 Fusion Model Training

The fusion model is trained with the script `scripts/train_fusion.py`. Several versions were trained on the same data so that they can be compared:

1. vision only (frozen MobileNetV3 and a classification head),
2. audio only (the AudioCNN structure),
3. naive fusion (both branches trained together from the start),
4. fusion with modality dropout only (0.3 and 0.5),
5. fusion with the training recipe described below.

Training recipe. The recipe was found after the naive fusion model failed (Section 5.7). It has two steps, and the order matters:

1. **Warm start the audio branch.** The weights of the trained audio-only model are copied into the audio branch of the fusion model, so the audio branch is already useful when fusion training starts.

2. **Modality dropout with p = 0.5.** During training, the image of each sample is replaced by a black image with probability 0.5. The model cannot rely on the camera alone and must use the audio as well.

Feature-vector model. For the hybrid fusion strategy (FB), a second version of the model takes the 576-number MobileNetV3 output directly instead of the image. The trained layers are copied over, and the script `scripts/prepare_fusion_fv.py` checks that both versions give the same prediction on every test sample, with real and with black images. The two versions agreed on 100% of the 103 test predictions, both with real and with black images (accuracy 0.718 and 0.398 for both). Computing the feature vectors took about 3.5 ms per frame on the laptop when frames are processed in batches (44 ms for a single frame, Section 5.9).

4.6 Live Deployment on the Raspberry Pi

For the first live run (Strategy A), moving from the replayed experiments to the real Raspberry Pi needed only two configuration changes and no change to the server code:

- the broker listens on all network interfaces (`0.0.0.0`) instead of only on the laptop itself, so that the Pi can reach it over the local network,
- the Pi client is given the laptop's IP address as broker address.

Only the start of the client loop is different. Table 4.3 shows the comparison.

Table 4.3: Differences between the replayed client and the Pi client.

	Replayed client	**Raspberry Pi client**
Audio source	WAV file from the dataset	USB microphone
Label source	dataset label	YOLOv8 on a live camera frame
Buffer, serialise, publish	identical	identical
Broker and server	identical	identical

Runtime vocabulary. The live camera sees things like people, cars and phones, which are COCO classes. The UrbanSound8K model knows classes like siren and jackhammer. The two sets of names have nothing in common, so at first none of the live samples could be used for training. The solution was an enrollment step: the Pi records for a few minutes, YOLO labels each tick, and the classes that appear at least 30 times become the vocabulary for this deployment. The resulting dataset is saved once and replayed by all three strategies, so that their numbers can be compared. For this, the servers got one extra option that loads the enrolled class list and test set instead of UrbanSound8K; the rest of the server code stayed the same. The audio model is warm-started from the UrbanSound8K model:

all convolution layers are copied and only the output layer is replaced with a new one of the right size.

Inference with a reject option. After training, the model can be copied back to the Pi and used on live audio. A plain softmax classifier always picks one of its known classes, even for a sound it has never heard. To avoid this, the Pi reports "unknown" when the highest probability is below a threshold. If several unknown sounds come in a row while the camera sees an object, the device reports that it has noticed something new, which in a real deployment would trigger a new enrollment and a new round of training.

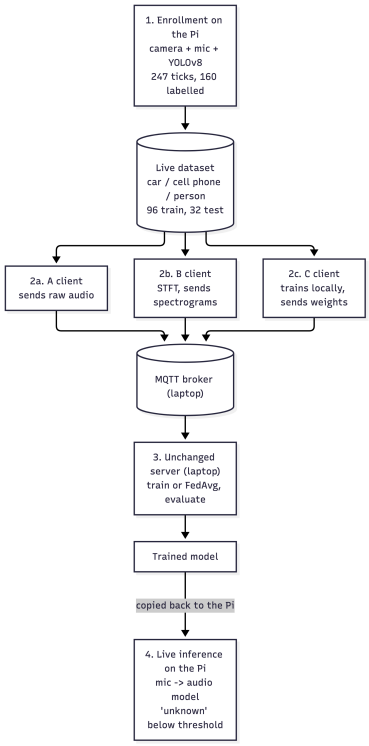


Figure 4.1: Live pipeline: enrollment on the Pi, replay by the A, B and C clients, training on the unchanged server, and live inference on the Pi.
Source: own work.

4.7 Problems and How They Were Solved

A broken microphone that reported no error. The microphone built into the webcam produced a constant signal at full scale. There was no driver error, and the device looked fine in the system. The problem was found by looking at the data itself: the RMS value of every clip was equal to the maximum possible value, which means the signal was constant and not sound. The webcam microphone was replaced with a separate USB microphone. Lesson: sensors should be checked by looking at their data, not only at their status.

TensorFlow and PyTorch in one process. On the Raspberry Pi, loading TensorFlow (for the audio model) and PyTorch (for YOLO) in the same process crashed with a segmentation fault, caused by a clash between their OpenMP libraries. This could not be fixed by changing the import order or the number of threads. Because enrollment only needs YOLO and inference only needs TensorFlow, they were split into separate programs.

Federated training made the model worse. The first federated runs used the same learning rate as the server (1e-3), and accuracy went down instead of up. This was solved with a learning-rate sweep (Section 5.4).

The fusion model ignored audio. Different training runs gave suspiciously identical accuracy values, and with black images the model gave the same answer for every clip. Section 5.7 explains how this was traced to an audio branch that had stopped learning.

YouTube download changes. YouTube stopped serving combined video and audio streams in the format the download script asked for, so many VGGSound clips failed to download. The script was changed to download the best low-resolution video stream and the best audio stream separately and let ffmpeg merge them.

Wrong frame count in early tests. An early test expected 49 frames per spectrogram. The correct value is 51 with centred frames (librosa default) and 48 without. The difference comes from padding at the edges of the clip (Section 2.1).

4.8 Order of Implementation

The system was not built in one go. Table 4.4 shows the order in which I built it, what worked and what did not work at first. The dates come from the version history of the repository.

Table 4.4: Order of implementation.

When	Step	What happened
June 2026	YOLOv8 labeller, STFT pipeline, first AudioCNN	worked; the labeller and the spectrogram shape were checked on test images and UrbanSound8K
July 2026	first Strategy A simulation with mocked messaging; first Strategy B with 8-bit spectrograms	worked as a prototype, but without a real broker the bytes were not real network traffic
early August 2026	first federated version with Flower; then Strategy A over a real MQTT broker and the first live run on the Raspberry Pi	Flower used a different transport than A and B, so the bytes were not comparable and I moved C to MQTT; the live run worked after replacing the broken webcam microphone (Section 4.7)
August 2026	Strategy B, then Strategy C with my own FedAvg over MQTT	B worked directly; C ran, but made the model worse until I lowered the local learning rate (Section 5.4)
mid-August 2026	fusion model, VGGSound data, FB and FC	the first fusion model ignored the audio; I found the cause with probes and fixed it with warm start and modality dropout (Section 5.7)
September 2026	live enrollment, Pi clients for A, B and C, live inference	worked on the Pi; TensorFlow and YOLO had to run in separate programs (Section 4.7)
October 2026	strategy, fusion and labelling experiments run again; latency and live-data training measured	needed because the result files of the first runs were not kept; the baseline model (July) and the live Pi results (September) were kept; some earlier fusion numbers could not be reproduced exactly (Section 5.7)

Source: version history of the project repository.

The Raspberry Pis were powered from the mains, as planned for this thesis, so battery operation and power consumption in watts were outside its scope; the computing time on the device is used as an indicator of the energy cost instead. The live tests in this thesis used one Raspberry Pi 5; the two-node comparison was done with two replayed nodes in software. Running federated training live between two physical Raspberry Pis, as originally planned, is the next step and is prepared for the defense demonstration (Section 7.3).

5 Results

This chapter presents what I measured. Each section shows a table or a figure and describes what it shows; what the results mean is discussed in Chapter 6. Every number comes from a result file in my repository, and Appendix A lists the commands for the main experiments. The strategy, fusion and labelling experiments were run again on 1 October 2026, so that every number can be traced to a file; the baseline model is from July and the live Pi results are from September 2026.

5.1 Building Blocks

5.1.1 YOLOv8 as a labeller

To check how good the automatic labels are, YOLOv8n was run on 150 VGGSound video thumbnails (30 per class) and its label was compared with the human label of the sound.

Table 5.1: YOLOv8n as an automatic audio labeller.

	Count	Share
YOLO label matches the sound label	40	26.7% of all, **85.1% of detections**
YOLO gives a different label (label noise)	7	4.7% of all, 14.9% of detections
YOLO detects nothing (sample skipped)	103	68.7%

Source: `data/vggsound/label_noise_results.json`.

When YOLO made a detection, it was correct in 85% of the cases, so about 15% of the automatic labels are wrong. The typical mistakes are between similar classes, such as motorcycle and car or dog and cat. In about two thirds of the frames YOLO found none of the five objects; these samples are skipped, which costs data but does not add wrong labels. Thumbnails are often title images rather than real scenes. During the live enrollment (Section 5.6) YOLO made a confident detection in 64.8% of the frames, although there any COCO class counted, not only these five. YOLOv8n took about 36 ms per image on the laptop and about 318 ms per frame on the Raspberry Pi 5.

5.1.2 Baseline AudioCNN

The AudioCNN was trained on UrbanSound8K folds 1–8 with fold 9 for validation. Training stopped after 21 epochs (early stopping), which took about 45 minutes on the laptop. On the unseen fold 10 (837 clips) the model reached a **test accuracy of 64.9%** and a macro F1 score of 0.649. Random guessing between ten classes would give 10%.

Table 5.2: Per-class results of the baseline AudioCNN on fold 10.

Class	Precision	Recall	F1
air conditioner	0.52	0.85	0.65
car horn	0.69	0.73	0.71
children playing	0.53	0.36	0.43
dog bark	0.75	0.62	0.68
drilling	0.74	0.82	0.78
engine idling	0.53	0.73	0.62
gun shot	0.68	0.94	0.79
jackhammer	0.99	0.69	0.81
siren	0.76	0.27	0.39
street music	0.62	0.68	0.65
macro average	**0.68**	**0.67**	**0.65**

Source: `data/urbansound8k/training_metrics.json`.

The weakest classes are siren and children playing. The confusion matrix shows that sirens are often mistaken for air conditioners, children playing and dog barks. Many siren recordings contain long stretches of traffic noise, and a single 500 ms window often does not contain the siren itself. The strongest classes, gun shot and jackhammer, have short and very characteristic patterns.

This model is the starting point for all strategy runs in stage one.

5.2 Strategy A: Centralized

Table 5.3: Strategy A per round.

Round	Samples in pool	Test accuracy	Server training time
1	1,000	0.638	3.6 s
2	2,000	0.681	7.3 s
3	3,000	0.644	10.0 s
4	4,000	0.638	12.2 s
5	5,000	0.612	14.5 s
6	6,000	0.591	18.2 s
7	7,000	0.639	22.0 s
8	8,000	0.664	23.1 s
9	9,000	0.671	26.3 s
10	10,000	0.661	30.1 s

Source: `results/strategy_a/run_20261001_160849/server.json`.

Each node streamed 5,000 samples in about 42 minutes (one sample every 500 ms) and uploaded them in 10 batches. Node A uploaded 69.24 MB and node B 69.06 MB of compressed raw audio, about 13.8 KB per sample. The byte counts recorded by the clients and by the server matched exactly, and no upload failed.

The server training time grew linearly, from 3.6 s in round 1 to 30.1 s in round 10, because the centralized server trains on everything it has ever received. The test accuracy moved

between 0.59 and 0.68 and ended at 0.661, close to the 0.649 of the starting model. It dipped in rounds 5 and 6 and recovered afterwards.

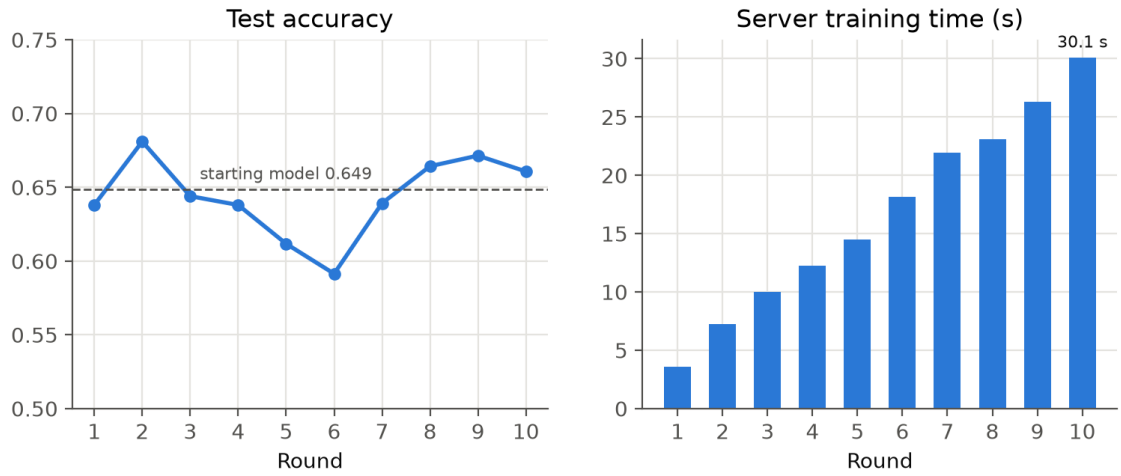


Figure 5.1: Test accuracy (left) and server training time (right) per round for Strategy A.
Source: own work.

5.3 Strategy B: Hybrid

Table 5.4: Strategy B, selected rounds.

Round	Test accuracy	Server training time
1	0.621	3.7 s
5	0.654	13.8 s
10	0.688	30.1 s

Source: `results/strategy_b/run_20261001_165113/server.json`.

Moving the STFT to the device reduced the upload to 51.01 MB (node A) and 51.07 MB (node B) per node, which is **26.2% less** than Strategy A. The accuracy moved between 0.62 and 0.70 and ended at 0.688, so there was no accuracy penalty. Each device spent 11.3 s in total computing spectrograms for its 5,000 samples, about 2.3 ms per clip.

The saving is modest because a raw 500 ms clip is 16 KB (8,000 samples × 2 bytes) and a 16-bit spectrogram is about 13 KB (51 × 128 × 2 bytes), so the two are already close in size. Section 5.10.2 derives the expected saving from these sizes and explains the difference from the measured value.

5.4 Strategy C: Federated and the Learning-Rate Sweep

The first federated run used a local learning rate of 1e-3, the same value the server uses in strategies A and B. The accuracy fell to 0.42 in the first round, far below the 0.649 of the starting model, and then jumped up and down. To find the cause, the local learning rate was varied while everything else stayed the same.

Table 5.5: Effect of the local learning rate on federated training.

Local learning rate	Accuracy range over 10 rounds	Final accuracy	Behaviour
1e-3	0.42–0.66	0.554	unstable, mostly far below the start
5e-4	0.51–0.70	0.607	good start, then collapses and oscillates
1e-4	0.62–0.70	0.699	mostly above the start, still swings
5e-5	0.65–0.69	0.676	stable, only round 1 marginally below the start

Source: the four `results/strategy_c/run_20261001_18*` folders.

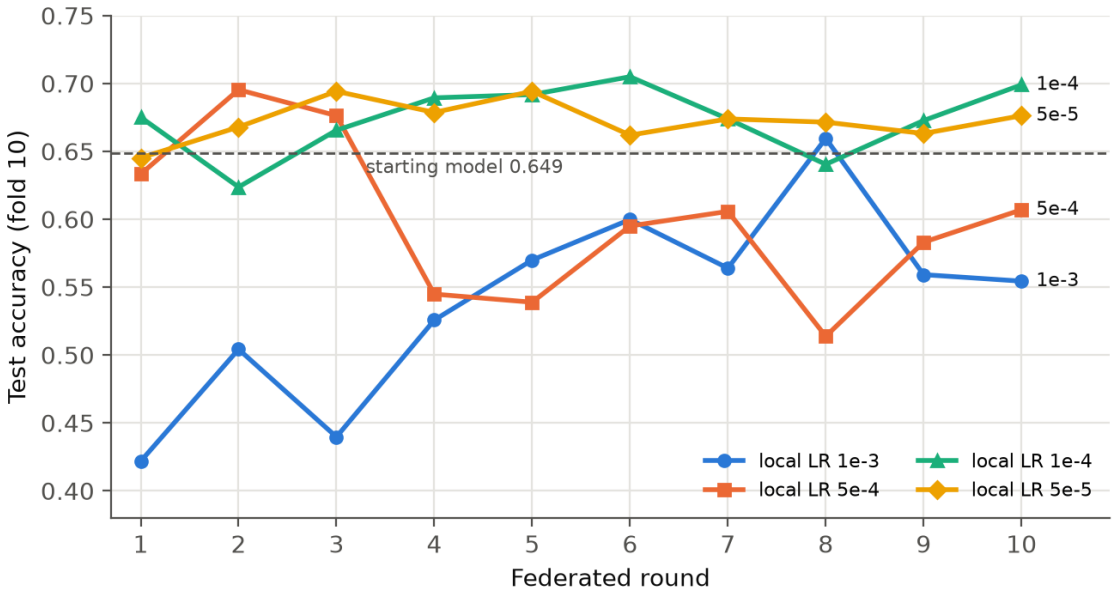


Figure 5.2: Test accuracy per federated round for four local learning rates (dashed: starting model).
Source: own work.

The behaviour improved step by step as the learning rate went down. At 5e-5 the accuracy stayed within a band of 0.05, and only round 1 dipped marginally below the starting model (0.645 against 0.649). At 1e-4 the best single round was slightly higher, but the swings were about 1.7 times as large. Because a deployed system cannot choose which round it stops at, 5e-5 was chosen as the setting for the comparison.

With this learning rate, each node uploaded 4.18 MB over ten rounds, about 418 KB per round, and the upload was the same for all four learning rates, because it depends only on the size of the model. The server only averaged the weights, which took 0.66–0.74 s per round. Each device spent 233 s in total on local training, about 23 s per round for 3,540 clips.

5.5 Comparison of the Three Strategies

Table 5.6: Comparison of strategies A, B and C.

	A Centralized	**B Hybrid**	**C Federated (LR 5e-5)**
Upload per node	69.2 MB	51.0 MB	4.18 MB
Upload compared with A	–	−26.2%	**16.5× less**
Accuracy range (final)	0.59–0.68 (0.661)	0.62–0.70 (0.688)	0.65–0.69 (0.676)
Device compute	0	11.3 s (STFT)	233 s (training)
Server compute per round	3.6 → 30.1 s, growing	3.7 → 30.1 s, growing	0.7 s, constant
Data leaving the device	raw audio	spectrograms	weights only

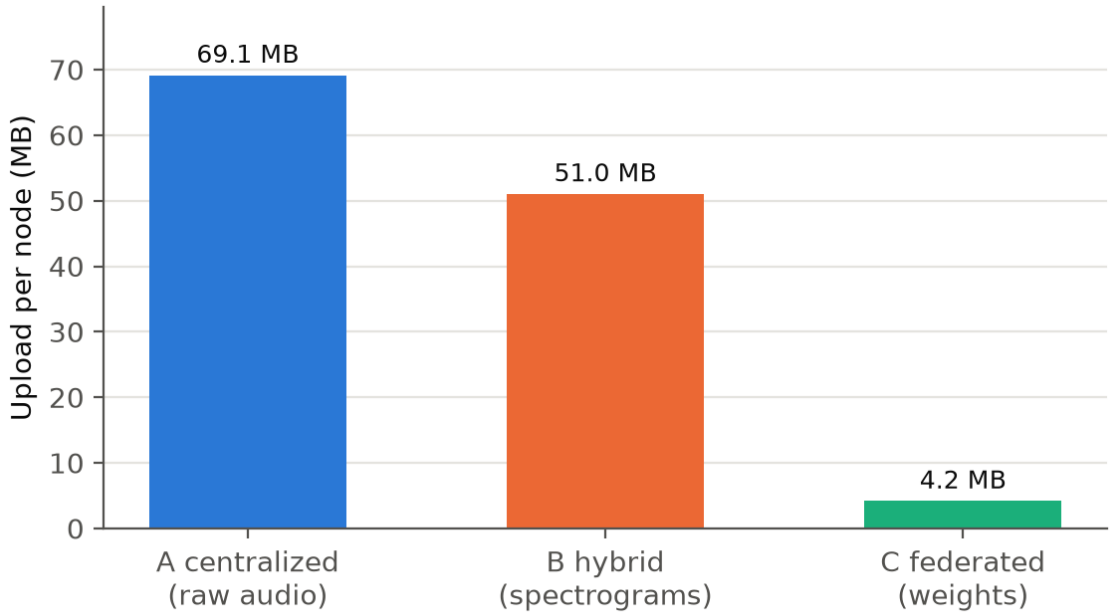


Figure 5.3: Upload per node for strategies A, B and C.
Source: own work.

From A to C, upload bandwidth falls from 69 to 51 to 4 MB per node and the server's work falls from a growing training job to a constant averaging step of under a second. The work on the device rises from 0 to 11 to 233 seconds. All three strategies end within 0.03 of each other in accuracy, and all three stay close to the starting model. In every run the sender's and the receiver's byte counts matched exactly and no upload failed.

5.6 Live Validation on the Raspberry Pi

5.6.1 Enrollment

During live enrollment the Raspberry Pi 5 recorded 247 ticks in about four minutes. YOLO produced a confident label for 160 of them (64.8%). The classes car (40 samples), cell phone (33) and person (55) appeared often enough and became the vocabulary. Rare

detections such as traffic light, bird and tv were dropped. The 128 samples were split into 96 for training and 32 for testing.

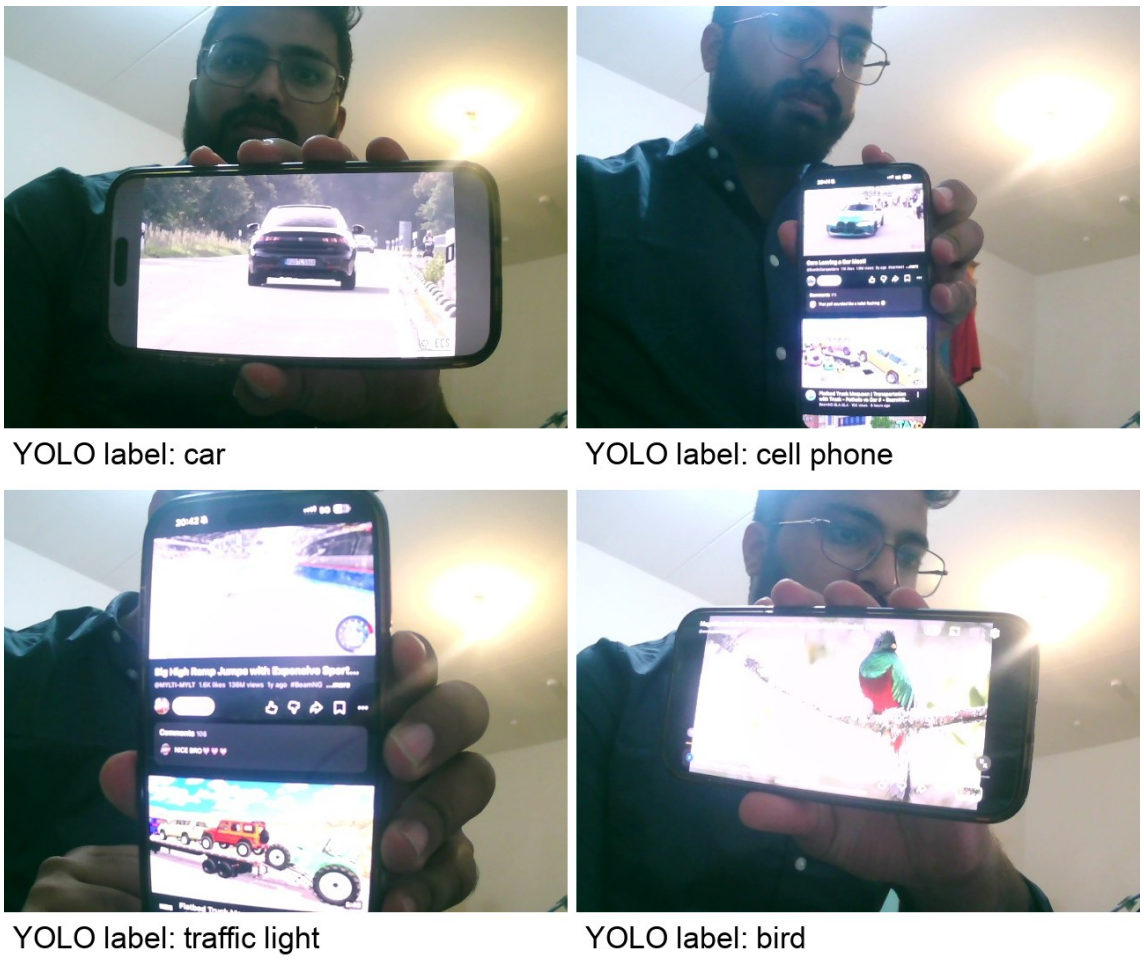


Figure 5.4: Frames from the live enrollment with the label YOLO assigned (top right and bottom left are wrong labels).
Source: own work.

During enrollment the objects were shown as videos on a phone screen held in front of the camera, and the phone's speaker played the matching sound. This made it possible to test classes like car indoors. It also produced live examples of label noise (Figure 5.4): YOLO sometimes labelled the phone itself ("cell phone") or a small object inside the video ("traffic light") instead of the object that made the sound.

5.6.2 Strategies A, B and C on the Pi

The same enrolled dataset was then replayed by the Pi clients of all three strategies over the local network to the unchanged laptop server.

Table 5.7: Live run on the Raspberry Pi 5 with 96 training samples and three classes (chance level 0.333).

	A Centralized	**B Hybrid**	**C Federated**
Upload	1,337 KB	979 KB (−26.7%)	1,245 KB (3 rounds, ~415 KB each)
Work on the Pi	–	STFT 1.1 s	STFT 1.1 s + training 52.7 s
Server time per round	2.3–3.0 s training	2.2–2.8 s training	0.09–0.18 s (one client, so no real averaging)
Final test accuracy	0.531	0.531	0.563
Raw audio kept on device	0	1,536 KB (spectrograms sent instead)	1,536 KB

Source: `data/pi_captures/results/live/`.

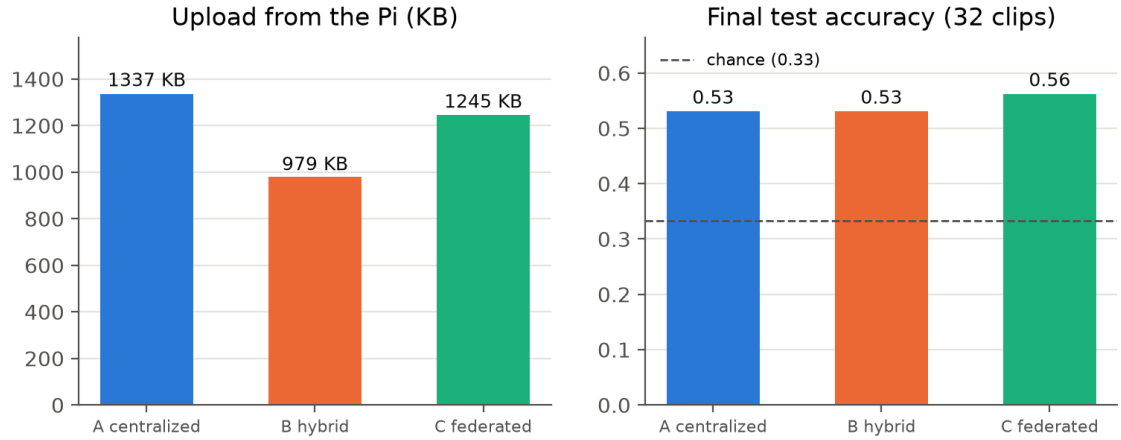


Figure 5.5: Upload and final accuracy of the three strategies on the real Raspberry Pi 5.
Source: own work, from the result files in `data/pi_captures/results/live/` and `results/demo/`.

With only 96 training samples the accuracy is modest, but clearly above chance. The pattern is the same as in the replayed experiments: B uploads less than A, C uploads a fixed amount per round, and the work on the device grows from A to C. With this small dataset, C uploads about as much as A, because three rounds of model weights are about as large as 96 audio clips. The server handled the data from the real Pi in the same way as the data from the replayed clients.

The live demonstrator will also be shown during the defense: the objects are chosen on the spot, the Pi enrolls them, the three strategies train on the new data, and the trained model is tried on live sound. The numbers of that run will differ from Table 5.7, because the objects, the room and the amount of data will differ, but the pattern should be the same.

For the defense demonstration, the live setup will be extended to two Raspberry Pis, as originally planned. Each Pi records and labels its own data, trains locally and sends only its weights to the laptop server, which averages them with FedAvg and sends the new model back to both Pis. This will show in real time what the two replayed nodes showed in

software in Sections 5.4 and 5.5. The software side stays the same; the second device will be added with only small changes (Section 7.3).

5.6.3 Which training settings work on the live data?

The live run used one setting (learning rate 1e-3, 8–10 epochs per round). To check whether this was a good choice, the recorded live dataset was trained again offline on the laptop with different settings: two learning rates, with and without the warm start from the UrbanSound8K model, 40 epochs, batch size 8 as on the Pi, and three repetitions with different random seeds. The audio is exactly what the Pi microphone recorded and the labels are exactly what YOLO assigned.

Table 5.8: Test accuracy on the live dataset (32 test clips, chance 0.33), mean ± standard deviation over three seeds.

Setting	after 1 epoch	after 5	after 10	after 20	after 40
Warm start, LR 1e-3	0.41 ± 0.03	0.41 ± 0.09	0.49 ± 0.06	0.50 ± 0.11	0.51 ± 0.03
Warm start, LR 1e-4	0.36 ± 0.03	0.39 ± 0.03	0.45 ± 0.05	0.47 ± 0.08	0.50 ± 0.03
From scratch, LR 1e-3	0.29 ± 0.03	0.31 ± 0.00	0.29 ± 0.03	0.29 ± 0.03	0.30 ± 0.01
From scratch, LR 1e-4	0.31 ± 0.00	0.31 ± 0.00	0.31 ± 0.00	0.31 ± 0.00	0.31 ± 0.00

Source: `data/pi_captures/results/live/training_sweep.json`.

Three things stand out. First, the warm start is essential: without it the model never got above chance, and at LR 1e-4 it predicted the same class for every clip in every run (10 of 32 = 0.31). With only 96 training clips, the convolution layers learned on UrbanSound8K are what makes learning possible at all. Second, more epochs help up to about 10–20 epochs, after which the accuracy levels off around 0.50. Third, the learning rate matters much less here than in federated training: both rates end at about the same level. The setting used in the live demo (LR 1e-3, about 10 epochs) was therefore a reasonable choice, and the 0.53–0.56 reached in the demo lies within the spread of this sweep. With 32 test clips, one clip is 3 percentage points, so differences smaller than about 0.1 should not be over-interpreted.

5.6.4 Live inference

The trained model was copied back to the Pi and run on live microphone audio. With a threshold of 0.95, 3 of 24 ticks were recognised as person and the rest were reported as unknown, which is the expected behaviour for a quiet room with sounds the model was not trained on.

5.7 The Fusion Model and the Blackout Test

The fusion experiments use the VGGSound subset with five classes (987 clips: 788 for training, 96 for validation, 103 for testing), so random guessing gives 20%. Each model was evaluated on the same 103 test clips, once with the real images (clean) and once with every image replaced by a black one (blackout).

The vision-only and audio-only models were trained once. The fusion models were trained twice: once with the default settings of my training script (up to 40 epochs, stopping after 8 epochs without improvement), and once for a fixed 25 epochs. In both cases the script keeps the weights of the epoch with the lowest validation loss. The dropout-0.3 model was only trained in the first run.

Table 5.9: Clean and blackout accuracy of the fusion experiments (103 test clips, chance 0.20).

Model	Run 1: clean	Run 1: blackout	Run 2: clean	Run 2: blackout
Vision only	0.806	0.204	(same)	(same)
Audio only	0.505	0.505	(same)	(same)
Naive fusion	0.767	0.204	0.738	0.204
Fusion + modality dropout 0.3	0.777	0.194	–	–
Fusion + modality dropout 0.5	0.680	0.175	0.748	0.126
Fusion + audio warm start + dropout 0.5	0.757	**0.359**	0.718	**0.398**

Source: `data/vggsound/fusion_blackout_results_early_stopping.json` and `data/vggsound/fusion_blackout_results.json`.

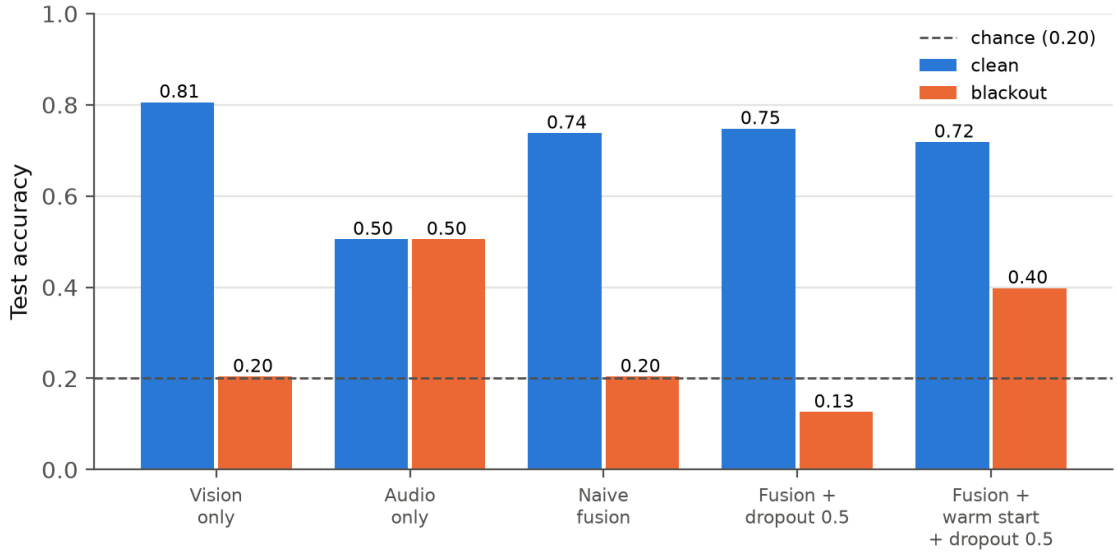


Figure 5.6: Clean and blackout accuracy for each model (run 2).
Source: own work.

The best epoch came very early. In both runs, the lowest validation loss was reached after only one or two epochs, so every fusion model kept those early weights, even when it trained for 25 epochs. The validation set has only 96 clips, and the pre-trained vision branch fits it almost at once. The two recipe results (0.359 and 0.398 in blackout) therefore come from nearly the same point in training and differ only by normal run-to-run variation. Training longer and keeping the last epoch instead of the best one was not tested.

Without the recipe, the camera failure is total. In blackout, the naive fusion model and both dropout-only models fell to chance level or below (0.13–0.20), exactly like the vision-only model. To check whether these models use the audio at all, two probes were run on every fusion model (Table 5.10).

Table 5.10: Does the fusion model listen? Blackout: number of different classes predicted when all images are black. Audio swap: real images, audio shuffled between test clips.

Model	Different classes predicted in blackout	Predictions changed when audio is swapped
Naive fusion	1 of 5	2 of 103
Fusion + dropout 0.3	2 of 5	2 of 103
Fusion + dropout 0.5	2 of 5	3 of 103
Fusion + warm start + dropout 0.5	**5 of 5**	**23 of 103**

Source: `data/vggsound/fusion_audio_probe.json`.

With black images the audio is the only input that still changes, yet the naive model predicted one single class for all 103 clips. Its blackout score of 0.204 is exactly the share of that class in the test set (21 of 103), not a sign of hearing anything. Swapping the audio between clips changed only 2 of 103 predictions. The naive model has learned to ignore the audio completely. Modality dropout alone did not change this: blacking out the image during training does not help if the audio branch never learned anything useful in the first place.

The full recipe made the model listen. With the audio branch warm-started from the trained audio-only model and modality dropout of 0.5, the model used all five classes in blackout, swapping the audio changed 23 of 103 predictions, and it kept 0.398 accuracy with a black camera, about twice the chance level and 79% of what the audio-only model achieves.

The price is clean accuracy. On clean data, the vision-only model was the best model (0.806). No fusion model beat it in these runs; the recipe model reached 0.718. Fusion

made the system robust to camera failure, but in this setup it did not make it more accurate when the camera works.

Comparison with the earlier run. An earlier version of this experiment, whose result files are not available, reported 0.847 clean and 0.551 blackout for the recipe model. That run used a different download of VGGSound (98 test clips, and a different set of bird clips) and its audio-only model was stronger (0.561 against 0.505 here). The qualitative result is the same in both runs: only the recipe model keeps working when the camera fails. The exact numbers are not, which shows how much a test set of about 100 clips can move.

5.8 Fusion Model under the Hybrid and Federated Strategies

The recipe model (0.718 clean, 0.398 blackout) was then used as the starting point for the hybrid (FB) and federated (FC) strategies, with two nodes that each hold half of the 3,152 training samples (1,576 each). FB streamed samples every 50 ms in batches of 250 and trained after every 500 new samples, for 6 rounds (3,000 samples in the pool at the end). FC ran 10 rounds with a local learning rate of 1e-4 and one local epoch per round. Both runs measured clean and blackout accuracy after every round.

Table 5.11: Fusion model under the hybrid and federated strategies.

	FB hybrid fusion	**FC federated fusion**
What is sent	spectrogram + feature vector (10.3 KB per sample)	weights only (1.44 MB per round)
Upload per node	16.2 MB (1,576 samples)	14.4 MB (10 rounds)
Clean accuracy (final)	0.67–0.85 (0.845)	0.79–0.88 (0.854)
Blackout accuracy (final)	0.49–0.60 (0.583)	0.43–0.67 (0.573)
Server compute per round	trains, 2.8 → 9.2 s, growing	averages, 0.2–0.3 s
Device compute	feature vector + STFT only	114 s local training per node

Source: `results/strategy_fb/run_20261001_193556/` and `results/strategy_fc/run_20261001_193733/`.

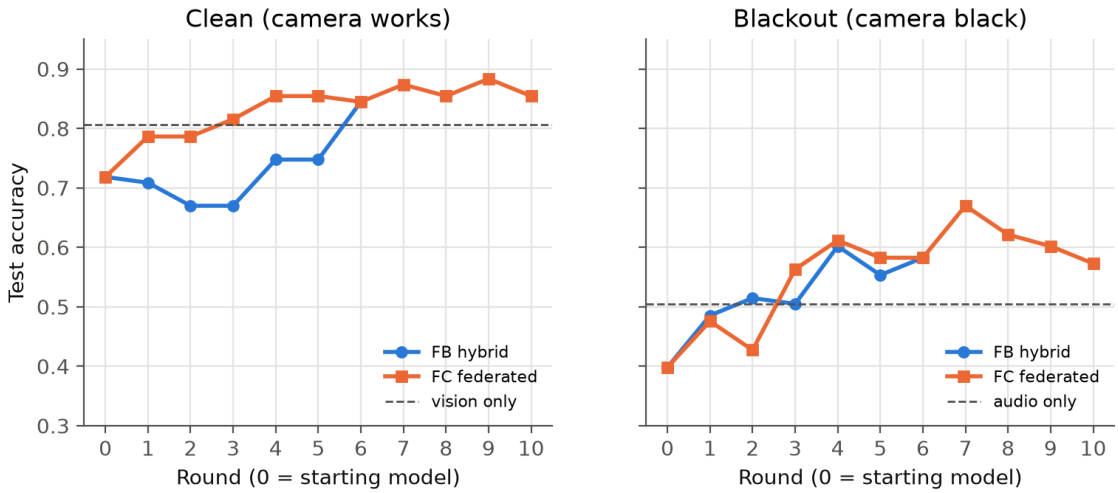


Figure 5.7: Clean (left) and blackout (right) accuracy per round for FB and FC.
Source: own work.

In Figure 5.7, round 0 is the starting model, and the dashed lines show the vision-only model (left) and the audio-only model (right). Both strategies improved the model in both conditions. On clean data, FB ended at 0.845 and FC at 0.854 (best round 0.883), both above the vision-only model (0.806). In blackout, both rose from 0.398 to a final 0.583 (FB) and 0.573 (FC), above the audio-only model (0.505); FC reached 0.670 in its best round. The ability to work without the camera did not get lost under continued training; it got stronger.

Part of this improvement is simply more training. FB and FC continue to train on the same kind of data as the starting model, so they effectively add more epochs. The vision-only and audio-only models did not get this extra training, so the comparison with them in the last rounds is not fully equal.

The federated upload per round was 1.44 MB, 3.45 times the 418 KB of the audio model. The ratio of the numbers they send is 389,893 to 111,370, which is 3.50. The upload in federated learning follows the size of the model almost exactly.

5.9 Latency

While I was planning this work, latency came up as an important point next to bandwidth, separated into inference (using the model) and training (improving the model). This section collects both.

5.9.1 Inference latency

Table 5.12 shows how long each step takes for one clip. The laptop values were measured with batch size 1 and 200 repetitions after a warm-up. The Raspberry Pi values come from the live run (Section 5.6).

Table 5.12: Latency per step for one clip.

Step	Laptop (median)	Raspberry Pi 5	Needed in
YOLOv8n, one camera frame	36 ms	318 ms	all strategies (labelling)
STFT, one 500 ms clip	0.65 ms	11 ms	A on the server; B and C on the device
AudioCNN, one prediction	5.4 ms	not measured	all strategies (inference)
MobileNetV3 feature vector, one frame	44 ms	not measured	FB and FC
Fusion head, one prediction	7.0 ms	not measured	FB and FC

Sources: `results/latency/inference_latency_MacBookAir.json` (laptop, except YOLO: `logs/rerun_20261001/12_label_noise.log`), `data/pi_captures/results/live/` (Pi: YOLO from the enrollment report, STFT = 1.06 s / 96 clips).

Two things follow from this table. First, **inference latency is the same in all three strategies.** By design, inference always runs on the device with the same model; the strategies only differ in where the model is trained. Second, the camera label is by far the slowest step on the Pi. YOLO uses 318 ms of the 500 ms tick, while the STFT that strategies B and C add on the device costs only 11 ms. Moving the STFT to the device therefore does not threaten the real-time loop.

5.9.2 Update latency

The strategies differ in how long it takes until a new, improved model exists. Table 5.13 compares this.

Table 5.13: Update latency of the three strategies.

	A Centralized	B Hybrid	C Federated
Work for one model update	server trains on everything received	server trains on everything received	each device trains on its own data, server averages
Time of that work	3.6 → 30.1 s, growing every round	3.7 → 30.1 s, growing every round	23 s per round on the laptop, 17.5 s on the Pi (96 clips), plus 0.7 s averaging
Grows with all data ever collected?	yes	yes	no
Time between new models in my runs	about 250 s	about 250 s	about 24 s

Sources: the run folders of Sections 5.2–5.4 and the live run in Section 5.6.

The interval of about 250 s in A and B follows directly from the settings: the server trains after every 1,000 new samples, and two nodes each deliver one sample every 500 ms, so

$$\frac{1000 \text{ samples}}{2 \text{ nodes} \times 2 \text{ samples/s}} = 250 \text{ s}$$

which matches the measured 2,503 s for 10 rounds. Most of this time is spent waiting for data, not training. In my federated runs the data was already on the devices, so a round took only about 24 s. In a real deployment, Strategy C would also have to wait for new recordings, but its training time per round stays constant, while the training time of A and B keeps growing with every round.

The latency on the Pi for the AudioCNN and the fusion model, as well as memory use, were not measured and are listed as future work.

5.10 Checking the Measurements against the Formulas

A measured number is easier to trust when it can be predicted on paper. In this section I take the formulas from Chapter 2, put in the settings from my code, and compare the prediction with what I measured. Where the two differ, I explain why. The STFT and FedAvg formulas come from the sources cited in Chapter 2 [1], [27]; the size, break-even and timing calculations are my own derivations from those formulas.

5.10.1 Spectrogram shape

The code (`src/edge/processing/stft.py`) uses $N = 8000$ samples, window $W = 400$, hop $H = 160$ and $N_{FFT} = 512$.

$$T = 1 + \left\lfloor \frac{N}{H} \right\rfloor = 1 + \left\lfloor \frac{8000}{160} \right\rfloor = 51 \text{ frames}$$

$$K = \frac{N_{FFT}}{2} + 1 = \frac{512}{2} + 1 = 257 \text{ frequency bins}$$

$$\text{overlap} = \frac{W - H}{W} = \frac{400 - 160}{400} = 60\%$$

The 257 bins are reduced to 128 mel bands. With the formula from Section 2.1, the highest frequency, 8 kHz, is $\text{mel}(8000) = 2595 \cdot \log_{10}(1 + 8000/700) \approx 2840$ mel, so the 128 bands would be about $2840/129 \approx 22$ mel apart (librosa's Slaney version spaces them slightly differently, with the same idea). The measured output shape is 51 × 128, as predicted.

5.10.2 Bytes per sample in strategies A and B

One raw clip in Strategy A is stored as 16-bit integers:

$$S_A = 8000 \times 2 \text{ B} = 16{,}000 \text{ B}$$

One spectrogram in Strategy B is stored as 16-bit floats:

$$S_B = 51 \times 128 \times 2 \text{ B} = 13{,}056 \text{ B}$$

Before compression, the expected saving of B over A is therefore

$$1 - \frac{S_B}{S_A} = 1 - \frac{13{,}056}{16{,}000} = 18.4\%$$

The measured upload per sample was 13,830 B for A and 10,208 B for B. The compression factor $c = \text{measured/uncompressed}$ is

$$c_A = \frac{13{,}830}{16{,}000} = 0.86$$

$$c_B = \frac{10{,}208}{13{,}056} = 0.78$$

So the measured saving is

$$1 - \frac{c_B \cdot S_B}{c_A \cdot S_A} = 1 - \frac{10{,}208}{13{,}830} = 26.2\%$$

The simple size formula predicts 18%; the measurement gives 26%. The extra 8 percentage points come from compression: spectrograms are smoother than raw audio and compress better (0.78 against 0.86). Both numbers are far from the large saving one might expect from "sending features instead of raw data", because a 500 ms spectrogram stored at 16 bit is almost as large as the 500 ms of audio it was made from.

5.10.3 Upload per round in federated learning

From Section 2.5.6, the upload per client and round is the number of parameters *P* sent times 4 bytes. For the AudioCNN with $P = 111{,}370$:

$$B_C \approx 111{,}370 \times 4 \text{ B} = 445{,}480 \text{ B} \approx 445 \text{ KB}$$

Measured: 4,183,866 B over 10 rounds, so 418 KB per round. The ratio measured / predicted is 0.94: zlib compression saves about 6%, more than the small JSON header adds.

For the fusion model, which sends $P = 389{,}893$ values (its 389,445 trainable parameters plus 448 batch-normalisation statistics):

$$B_{FC} \approx 389{,}893 \times 4\ \text{B} = 1{,}559{,}572\ \text{B} \approx 1.56\ \text{MB}$$

Measured: 1.44 MB per round (ratio 0.92). The ratio between the two models is predicted by the parameter counts alone:

$$\frac{B_{FC}}{B_C} \approx \frac{389{,}893}{111{,}370} = 3.50$$

$$\text{measured: } \frac{1.441\ \text{MB}}{0.418\ \text{MB}} = 3.45$$

5.10.4 When does federated learning save bandwidth?

In Strategy A the upload grows with the number of samples n collected between two rounds; in Strategy C it does not. Setting the two equal gives the break-even point:

$$n \cdot 13{,}830\ \text{B} = 418{,}387\ \text{B} \quad \Rightarrow \quad n^* \approx 30\ \text{samples per round}$$

If a device records more than about 30 samples between two federated rounds (15 seconds of labelled audio at one sample per 500 ms), federated learning uploads less than centralized training. In the replay experiments each node contributed 500 samples per round of A, so C was 16.5 times cheaper:

$$\frac{69.15\ \text{MB}}{4.18\ \text{MB}} = 16.5$$

The live Pi run confirms the formula from the other side. There, the 96 samples were spread over three rounds, on average 32 new samples per round, close to n^*, and A and C uploaded almost the same amount (1,337 KB against 1,245 KB, Table 5.7).

5.10.5 Bytes per sample in the hybrid fusion strategy (FB)

FB sends a 16-bit spectrogram and a 576-number feature vector stored as 16-bit floats:

$$S_{FB} = 13{,}056\ \text{B} + 576 \times 2\ \text{B} = 14{,}208\ \text{B}$$

Measured: 10,302 B per sample (compression factor 0.73). The feature vector adds only 1,152 B before compression, about 9% on top of the spectrogram, while a JPEG frame of the same scene would add 24–66 KB (Section 3.6.3).

5.10.6 Growth of server training time in Strategy A

The centralized server trains one epoch on its whole pool every round. If one sample costs k seconds to train, the time in round r with 1,000 new samples per round is

$$t_r = k \cdot 1000 \cdot r$$

and the total server time after R rounds is

$$T_R = \sum_{r=1}^{R} k \cdot 1000 \cdot r = k \cdot 1000 \cdot \frac{R(R+1)}{2}$$

The measured times give $k \approx 3.0$ ms per sample (between 2.9 and 3.7 ms in every round). With $k = 3.04$ ms the formula predicts $t_{10} = 30.4$ s (measured 30.1 s) and $T_{10} = 3.04 \text{ ms} \times 1000 \times 55 = 167$ s (measured 167.3 s). The time per round grows linearly and the total grows with the square of the number of rounds. In Strategy C, the server time per round stays constant (0.7 s), because averaging two models does not depend on how much data the clients have.

5.10.7 FedAvg weights in the experiments

Node A held $n_A = 3{,}540$ clips and node B $n_B = 3{,}539$. The FedAvg weights are

$$\frac{n_A}{n} = \frac{3{,}540}{7{,}079} = 0.50007$$

$$\frac{n_B}{n} = \frac{3{,}539}{7{,}079} = 0.49993$$

With an even split, FedAvg is almost exactly the plain mean of the two models. This also shows that the instability at high learning rates (Section 5.4) cannot come from one node dominating the average.

5.10.8 A blackout score that looks like chance

The test set has 21 bird, 21 car, 20 cat, 21 dog and 20 motorcycle clips. A model that always predicts one of the 21-clip classes scores

$$\frac{21}{103} = 0.204$$

which is exactly the blackout score of the vision-only and the naive fusion model. Together with the probe in Table 5.10 (one predicted class in blackout), this shows that these models do not guess randomly in the dark; they always give the same answer.

5.10.9 Summary

Table 5.14: Predictions from the formulas compared with the measurements.

Quantity	Formula predicts	Measured	Difference explained by
Spectrogram shape	51 × 128	51 × 128	–
B saving over A	18.4%	26.2%	spectrograms compress better
C upload per round	445 KB	418 KB	zlib compression
FC upload per round	1.56 MB	1.44 MB	zlib compression
FC / C upload ratio	3.50	3.45	–
Break-even samples per round	30	live run with 32: A ≈ C	–
A server time, round 10	30.4 s	30.1 s	–
A server time, total	167 s	167.3 s	–
Blackout score of a constant model	0.204	0.204	–

Every measured quantity is within a few percent of the prediction, or the difference has a clear cause. This makes it unlikely that the measurements contain a hidden error.

6 Discussion

In this chapter I explain what the results from Chapter 5 mean. I answer each research question, look at privacy, compare my results with related work and with what I predicted in my exposé, and then list the limitations. The chapter ends with a personal reflection on what was easy, what was hard and what I would do differently.

6.1 RQ1: How Do the Three Strategies Compare?

The results show a clear trade-off. Moving work from the server to the device reduces upload and server load and improves privacy, and the price is paid by the device. Accuracy was not part of the trade-off in my experiments: all three strategies ended at about the same accuracy.

Bandwidth. Strategy C needed 16.5 times less upload than Strategy A. The reason is simple: A and B send data, C sends a model. The data grows with every new recording, but the model always has the same size. The formula from Section 2.5.6 predicts about 445 KB per round for my audio model, and I measured 418 KB (zlib compresses the weights a little).

Server load. In Strategy A the server trains again on everything it has ever received, so every round takes longer than the last. In a system that runs for months this would grow without limit. In Strategy C the server only averages two models, which took less than a second per round and does not depend on how much data the devices have.

No cloud needed. All server work ran on an ordinary laptop (Apple M1, 8 GB RAM), so a cloud server was not necessary for this scale. A cloud server would add internet delay to every message, but it would not change the byte counts, because the same messages are sent either way. With many more devices, the growing training time of Strategy A would be the first reason to move the server to the cloud; the averaging in Strategy C would stay cheap.

Device workload. In Strategy C the device does about 20 times more work than in Strategy B: 233 s of training against 11.3 s of STFT over the whole run. In the replayed experiments the two "devices" were processes on my laptop, so these are laptop times. On the real Raspberry Pi 5, one federated round on the 96 live samples took about 17.5 s of

training (Section 5.6). That is fine for a system that trains every few minutes, but on weaker devices or with bigger models the device could become the bottleneck.

Why accuracy hardly changed. The accuracy stayed close to the starting model in all strategies. This can be explained and is not a failure: the starting model had already been trained on the same kind of data, so more of the same could not teach it much. These experiments measure the cost of each strategy. The experiments where accuracy really moves are the learning-rate sweep and the blackout tests.

The hybrid strategy. Moving the STFT to the device saved 26% of the upload. It sounds like "sending features instead of raw data" should save a lot, but Section 5.10.2 shows why it does not here: a 51 × 128 spectrogram at 16 bit is 13 KB, almost as big as the 16 KB of audio it comes from. A simple size calculation predicts 18%, and the smoother spectrograms compress a bit better, which gives the rest. Preparing data on the device only saves real bandwidth when the result is much smaller than the input, for example a 576-number feature vector instead of a camera image.

6.2 RQ2: How Does FedAvg Combine the Weights, and When Does It Match Centralized Training?

FedAvg combines the devices' models by averaging every weight array element by element, weighted by each device's share of the training samples (Sections 2.5.2 and 2.5.3). The weights do not cancel out, because every device starts each round from the same shared model.

FedAvg matched centralized training only after I reduced the local learning rate by a factor of 10 to 20 (1e-4 or 5e-5; I chose 5e-5 because it was the most stable). This happened even though both nodes had evenly split data, so it was not the usual explanation of different data on the clients (non-IID data) [2], [3].

The explanation is the starting point. The model was already trained, so it sat near the bottom of a valley in the loss landscape. With a large learning rate, each client takes a full epoch of big steps on its own data and climbs some distance up the side of the valley. The two clients climb in slightly different directions, and the average of two points high up on different slopes is not at the bottom. With small steps, both clients stay close to the starting point and their average stays near the bottom.

In short: **a model that is already trained needs gentler local updates under FedAvg than under centralized training.** FedProx tackles the same problem with its extra penalty term [4]. This matters for any system that, like mine, keeps improving a deployed model with federated learning.

6.3 RQ3: Does Fusion Survive When the Camera Fails?

Only the right training makes a fusion model listen. The fusion model trained the obvious way was deaf: with a black camera it predicted the same class for every clip, and swapping its audio changed only 2 of 103 answers. This matches what Wang et al. describe: the input that learns fastest takes over the training [21]. The pre-trained vision branch explained the labels from the first epoch, the loss became small, and almost no learning signal reached the untrained audio branch. The fusion head learned to ignore it.

The order of the two fixes matters. Modality dropout alone did not help: it makes the model look at the audio when the image is missing, but if the audio branch produces nothing useful, there is nothing to look at. Warm-starting the audio branch first gives it useful features from the start; modality dropout then teaches the head to use them. Only this combination gave a model that used all five classes in blackout and reacted to the audio.

Graceful degradation. In blackout, the vision-only model fell to chance level, while the recipe model kept 0.398, about twice chance. After further training under FB and FC it ended at 0.58 (FB) and 0.57 (FC) in blackout, above the audio-only model (0.505); FC reached 0.67 in its best round. Instead of failing completely when the camera stops working, the system falls back to hearing. This is what I set out to show.

Robustness first, accuracy second. Right after fusion training, the recipe model was less accurate on clean data than vision alone (0.718 against 0.806). Only after further training under FB and FC did it pass the vision-only model (final 0.845 for FB and 0.854 for FC). Part of this gain is extra training that the single-sense models did not get, so the safe conclusion is: fusion with the recipe makes the system robust to camera failure, and with enough training it does not have to cost accuracy when the camera works.

Federated cost follows model size. The federated upload per round grew from 418 KB for the audio model to 1.44 MB for the fusion model, a factor of 3.45, almost exactly the ratio of their parameter counts (3.50). With the amount of data I used, FB and FC cost about the

same (16.2 against 14.4 MB per node). With ten times more data, FB would send ten times more, while FC would send exactly the same.

6.4 Privacy

In Strategy A, raw audio leaves the device. Raw audio can contain speech and is the most sensitive form of the data. In Strategy B only spectrograms are sent. They still contain much of the sound: an approximate waveform can be rebuilt from a spectrogram with the Griffin–Lim algorithm [33], although the mel compression and the missing phase make the result rough. In Strategy C only model weights are sent. Camera images never leave the device in any of my strategies, and in the fusion strategies only a 576-number feature vector is sent instead of the image.

Federated learning is not a complete privacy solution. Attacks that rebuild training data from gradients [7] are known. My claim is therefore limited: in Strategy C, raw audio never leaves the device. Secure aggregation [8] or differential privacy would be needed for stronger guarantees.

6.5 Comparison with Related Work

Table 6.1: Comparison with related work.

Work	Setting	Main result	Relation to my thesis
McMahan et al. [1]	FedAvg in simulation	FedAvg needs far fewer communication rounds than plain federated SGD	same algorithm; I measured it over a real MQTT network
Ridolfi et al. [6]	Flower on Raspberry Pi	federated learning works on Raspberry Pi	confirmed; I also compared it with centralized and hybrid training
Aytar et al. [14]	SoundNet, vision teaches sound	sound features can be learned from unlabelled video	same idea, used here for continuous training on edge devices
Wang et al. [21]	multimodal training	multimodal models often do worse than their best single input	I saw the same problem; warm start and modality dropout fixed it

6.6 Comparison with My Exposé

6.6.1 Predictions versus results

In my exposé I made a table with what I expected from each strategy. Table 6.2 puts those expectations next to what I actually found.

Table 6.2: My expectations from the exposé compared with the results.

Aspect	What I expected (exposé)	What I found	Verdict
Bandwidth	A high, B medium, C low	69.2 / 51.0 / 4.18 MB per node	confirmed, but B was closer to A than I expected
Privacy	A low, B medium, C high	A sends raw audio, B spectrograms, C only weights	confirmed (by design, not measured as a number)
Accuracy	A best (most data), B good, C variable	all three ended within 0.03 of each other	not confirmed: with even data, C matched A once the learning rate was right
Edge power use	A low, B low–medium, C high	device time 0 / 11 / 233 s (plug-in powered, watts out of scope)	confirmed as computing time
Implementation effort	A simple, B medium, C complex	same order (Section 6.8)	confirmed
Main challenge of C	client drift from non-IID data	instability from a too-large learning rate, even with even data	partly different: the problem appeared earlier than expected

The biggest surprise was accuracy. I expected centralized training to win because the server sees all the data. In my experiments it did not, because the model was already trained and both nodes had the same kind of data. With very different data on the nodes, I would still expect centralized training to have an advantage.

6.6.2 The five metrics

The exposé named five metrics, and latency and memory were added during planning. Table 6.3 shows how far I could measure each one.

Table 6.3: The five metrics from the exposé and how far they were measured.

Metric (exposé)	Planned measurement	What I measured	Status
Accuracy	precision, recall, F1 on a shared test set	accuracy for all runs; per-class precision, recall and F1 for the baseline (Table 5.2)	done
Privacy	data-flow check; spectrogram reversibility	data flow checked for all strategies; reversibility discussed from literature (Section 6.4)	partly
Power consumption	USB power meter in watts	device computing time as a substitute	out of scope: plug-in powered devices
Bandwidth	MB per round	bytes counted at sender and receiver for every run	done
Model convergence	accuracy per round, FedAvg vs. baseline, non-IID	accuracy per round for every run; learning-rate sweep; non-IID not tested	partly
Latency and memory (added during planning)	inference vs. inference plus training	inference latency per step on the laptop, YOLO and STFT on the Pi, update latency per strategy (Section 5.9); memory not measured	partly

6.6.3 Changes to the plan

Several things turned out differently from the exposé plan. Table 6.4 lists them with the reason.

Table 6.4: Differences between the exposé plan and the final work.

Exposé plan	What I did	Why
Two Raspberry Pi 4 devices in different places	one Raspberry Pi 5 for the live tests; two replayed nodes on the laptop for the comparison	a fair comparison needs identical data for all strategies (Section 3.7.1); the live two-Pi run is planned for the defense demonstration
Camera Module 3 and I2S microphone	USB webcam and separate USB microphone	simpler to set up; the webcam's own microphone turned out to be broken (Section 4.7)
TensorFlow Lite on the Pi	full TensorFlow on the Pi	Strategy C needs training on the device, which TensorFlow Lite does not support well
Flower framework	own FedAvg over MQTT	one transport for all strategies keeps the byte counts comparable (Section 3.3)
Strategy A sends raw audio and video	raw audio only; images never leave the device	images are only needed for the label; sending them would add privacy risk for nothing
Non-IID data from different environments	evenly split data	the comparison came first; different data on two Pis in two labs is the next step (Section 7.3)
Power in watts	computing time	the Pis were plug-in powered, so battery operation and power measurement were outside the scope of this thesis

Exposé plan	What I did	Why
Local PC as server, AWS or Azure free tier as fallback	the laptop only; no cloud server	the laptop was enough: averaging took 0.7 s per round and server training at most 30 s
Select and implement one strategy	all three implemented, recommendation per situation	the results show that the best choice depends on the situation (Section 7.2)

6.7 Limitations and Threats to Validity

- **One test fold.** The UrbanSound8K results use fold 10 as the only test fold. Ten-fold cross-validation would give an average and a spread. The federated learning rate (Section 5.4) was also chosen by looking at the fold-10 accuracy, without a separate validation fold, so the result for 5e-5 is slightly optimistic.
- **Small fusion test set and no repeated runs.** The fusion test set has 103 clips, so one clip is about one percentage point, and an earlier run of the same experiment gave quite different numbers (Section 5.7). Each fusion model was trained at most twice, with different settings but not with several seeds; several random seeds would give a mean and a spread.
- **Evenly split data.** The two nodes had almost the same mix of classes. In a real deployment, devices in different places hear different things (non-IID data), which makes federated learning harder.
- **VGGSound images favour vision.** VGGSound clips are videos of the labelled object, so the object is usually easy to see. In real street scenes the camera view would be worse, and audio would matter more.
- **Black is not fog.** The blackout test is the extreme case of losing the camera completely. Real fog or darkness makes the image worse step by step.
- **Small live dataset and one Pi.** The live tests used one Raspberry Pi 5 and 96 training clips from about four minutes of enrollment. The results with two edge nodes come from the replayed software setup; the live run with two physical Pis is planned for the defense demonstration.
- **Phone videos in the live demo.** The live enrollment used videos played on a phone instead of real cars or animals. The pipeline is the same, but real outdoor recordings would contain more background noise.
- **Two clients.** FedAvg works the same way for any number of clients, but I did not measure effects of scale.

- **Plug-in powered devices.** The Pis were powered from the mains, so battery life and power in watts were outside the scope of this thesis; computing time is used as an indicator of the energy cost.
- **Not measured.** Memory use, the model inference latency on the Pi itself, and model freshness (how quickly each strategy learns a new object) were planned but not measured.
- **Baseline model.** The baseline audio model reaches 64.9%. Better audio models exist, but my thesis compares strategies, and all strategies start from the same model.

6.8 Personal Reflection

Strategy A was the easiest. It needs the least code on the edge device. The Pi only records the sound, gets the label from the camera and sends the raw data quickly to the central server. The server trains the model and sends it back, and the Pi then uses it to decide what it hears, for example "cat". Because the server does all the heavy work, there were few things that could go wrong, and when something did go wrong it was easy to find. That is also why I built it first, as the baseline for the other two.

Strategy C was the hardest. Here the Pi does not only collect data but also has to process it and train its own copy of the model, so there is much more code on the edge side. The server has to work in rounds, wait for both Pis, ignore late messages from an old round and send the shared model back. On top of that, I had to understand the formulas for averaging model weights (Section 2.5), which was not easy for me. And when everything finally ran, the model became worse instead of better. The reason turned out to be the learning rate, not a bug in the code.

The question about the weights. My supervisor asked me whether the weights of two different Pis would cancel each other out when they are averaged, so that the result is useless. My first ideas were about timing: maybe the weights must arrive at the server at the same moment, or the clocks of the Pis must be synchronised, or the server must keep the data of each Pi separately. When I read the FedAvg paper [1] and worked through the example in Section 2.5.3, one matrix element at a time, I understood the real reason why they do not cancel: in every round, both Pis start from exactly the same shared model, so each position in their matrices means the same thing. My timing idea was not completely wrong, though: the server does wait until both Pis have sent their weights for the same round before it averages them.

Difficulties and what I learned from them.

- The microphone built into the webcam was broken. The system showed it as working, but it only recorded a constant signal. I learned to check a sensor by looking at what it records, not only at its status.
- Understanding federated learning took me a long time, especially what happens to the weight matrices when they are averaged. Working through a small example by hand helped me most.
- The live tests used only one Raspberry Pi, and the live dataset was small: about four minutes of enrollment gave 96 training clips. This was enough to show that the pipeline works on real hardware, but not enough to train a strong model. I plan to show the live run with two Raspberry Pis in the defense demonstration.
- During the thesis period I had serious health problems, which slowed my work and created a lot of time pressure. Because of this, some planned parts, such as the experiment with different data on each device and the model-freshness experiment, could not be done within the thesis and are left for future work.

Which strategy I would choose. For now, I would use Strategy A, because it is fast, efficient for the device and the easiest to implement. For the future, I would move to Strategy C. A robot in the real world keeps meeting new sounds and new objects, and with federated learning it can learn from this new data on the device and keep its model up to date, while the raw data stays private.

7 Conclusion and Future Work

7.1 Summary

In this thesis I built an edge system in which a robot-like device, a Raspberry Pi with a camera and a microphone, collects sounds and labels them with the objects the camera sees. With this data I trained a machine learning model to recognise objects by their sound, and later by sound and image together. I then tried three different strategies for where the training happens, measured them with the same metrics and compared the trade-offs between them. I built the system step by step, and I learned a lot from the mistakes I made along the way.

The main findings are:

- **Accuracy:** all three strategies reached about the same accuracy.
- **Bandwidth:** moving work to the device reduced the upload per device from 69.2 MB (centralized) to 51.0 MB (hybrid) and 4.18 MB (federated), about 16 times less for federated learning than for centralized training.
- **Workload:** in the centralized strategy the server's training time grows with every round, while in federated learning the server only averages and the work moves to the device.
- **Learning rate:** federated learning matched centralized training only after its local learning rate was made much smaller (from 1e-3 to 5e-5).
- **Hybrid strategy:** sending spectrograms instead of raw audio saved only about a quarter of the bandwidth, because a spectrogram is almost as large as the sound it is made from.
- **Fusion:** a model that combines sound and camera images kept recognising objects when the camera was blacked out (0.40 accuracy, and about 0.58 after further training), while a camera-only model fell to chance (0.20).
- **Real hardware:** the complete system also ran live on a Raspberry Pi 5.

7.2 Recommendation

My original plan was to select one strategy. The results show that the best choice depends on the situation:

Table 7.1: Recommended strategy by situation.

Situation	Recommended strategy
Very weak device, fast and cheap network, privacy not critical	**A, centralized**
Device can compute spectrograms but cannot train	**B, hybrid**: reduces the exposure of raw audio, but saves little bandwidth
Limited or expensive network, privacy matters, device can train	**C, federated**
Large or growing amounts of data	**C**, because its upload does not grow with the data

For now, I would use Strategy A, because it is fast, efficient for the device and the easiest to implement. For the future, I recommend Strategy C: a robot in the real world keeps meeting new sounds and objects, and with federated learning it can learn from this new data on the device and keep its model up to date, while the raw data stays private. The local learning rate must be chosen carefully. Strategy B reduces the exposure of raw audio, because only spectrograms leave the device, but a spectrogram is not encryption and it saves only about a quarter of the bandwidth.

7.3 Future Work

1. **Model freshness.** Measure how fast each strategy learns when a new object appears. This is where federated learning is expected to have its biggest advantage.
2. **Latency, memory and battery operation on the Pi.** Measure the inference time and the memory use of each strategy on the device itself. For battery-powered robots, also measure the energy per round, which was outside the scope here because the Pis were plug-in powered.
3. **Two Pis in two places.** Run the federated rounds live between two Raspberry Pis, as originally planned; this is prepared for the defense demonstration. Placed in two different labs, each Pi hears different sounds, which also gives real cases of different data on each device (non-IID data).
4. **Better evaluation.** Use more test data for the fusion model and ten-fold cross-validation for UrbanSound8K, so that the results get an average and a spread instead of a single value.
5. **Stronger privacy.** Add secure aggregation or differential privacy to Strategy C, so that even the shared weights reveal as little as possible about the data.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR vol. 54, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] P. Kairouz et al., "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021, doi: 10.1561/2200000083.

[3] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin and V. Chandra, "Federated learning with non-IID data," arXiv:1806.00582, 2018. [Online]. Available: https://arxiv.org/abs/1806.00582

[4] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html

[5] D. J. Beutel et al., "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020. [Online]. Available: https://arxiv.org/abs/2007.14390

[6] L. Ridolfi, D. Naseh, S. S. Shinde and D. Tarchi, "Implementation and evaluation of a federated learning framework on Raspberry PI platforms for IoT 6G applications," *Future Internet*, vol. 15, no. 11, art. 358, 2023, doi: 10.3390/fi15110358.

[7] L. Zhu, Z. Liu and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/60a6c4002cc7b29142def8871531281a-Abstract.html

[8] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1175–1191, doi: 10.1145/3133956.3133982.

[9] J. Salamon, C. Jacoby and J. P. Bello, "A dataset and taxonomy for urban sound research," in *Proc. 22nd ACM Int. Conf. Multimedia*, 2014, pp. 1041–1044, doi: 10.1145/2647868.2655045.

[10] K. J. Piczak, "ESC: Dataset for environmental sound classification," in *Proc. 23rd ACM Int. Conf. Multimedia*, 2015, pp. 1015–1018, doi: 10.1145/2733373.2806390.

[11] S. Hershey et al., "CNN architectures for large-scale audio classification," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 131–135, doi: 10.1109/ICASSP.2017.7952132.

[12] B. McFee et al., "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 18–24, doi: 10.25080/Majora-7b98e3ed-003.

[13] H. Chen, W. Xie, A. Vedaldi and A. Zisserman, "VGGSound: A large-scale audio-visual dataset," in *Proc. IEEE ICASSP*, 2020, pp. 721–725, doi: 10.1109/ICASSP40776.2020.9053174.

[14] Y. Aytar, C. Vondrick and A. Torralba, "SoundNet: Learning sound representations from unlabeled video," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 29,

2016. [Online]. Available: https://papers.nips.cc/paper_files/paper/2016/hash/7dcd340d84f762eba80aa538b0c527f7-Abstract.html

[15] A. Owens, J. Wu, J. H. McDermott, W. T. Freeman and A. Torralba, "Ambient sound provides supervision for visual learning," in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, vol. 9905. Cham, Switzerland: Springer, 2016, pp. 801–816, doi: 10.1007/978-3-319-46448-0_48.

[16] R. Arandjelović and A. Zisserman, "Look, listen and learn," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 609–617, doi: 10.1109/ICCV.2017.73.

[17] D. Rolnick, A. Veit, S. Belongie and N. Shavit, "Deep learning is robust to massive label noise," arXiv:1705.10694, 2017. [Online]. Available: https://arxiv.org/abs/1705.10694

[18] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[19] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023, AGPL-3.0. [Online]. Available: https://github.com/ultralytics/ultralytics

[20] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324, doi: 10.1109/ICCV.2019.00140.

[21] W. Wang, D. Tran and M. Feiszli, "What makes training multi-modal classification networks hard?," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 12692–12702, doi: 10.1109/CVPR42600.2020.01271.

[22] X. Peng, Y. Wei, A. Deng, D. Wang and D. Hu, "Balanced multimodal learning via on-the-fly gradient modulation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 8228–8237, doi: 10.1109/CVPR52688.2022.00806.

[23] N. Neverova, C. Wolf, G. Taylor and F. Nebout, "ModDrop: Adaptive multi-modal gesture recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 38, no. 8, pp. 1692–1706, 2016, doi: 10.1109/TPAMI.2015.2461544.

[24] A. Banks, E. Briggs, K. Borgendale and R. Gupta, Eds., "MQTT Version 5.0," OASIS Standard, Mar. 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[25] R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," *J. Open Source Software*, vol. 2, no. 13, art. 265, 2017, doi: 10.21105/joss.00265. [Online]. Available: https://mosquitto.org

[26] V. Y.-S. Chan, C. T. Jin and A. van Schaik, "Neuromorphic audio–visual sensor fusion on a sound-localizing robot," *Frontiers in Neuroscience*, vol. 6, art. 21, 2012, doi: 10.3389/fnins.2012.00021.

[27] M. Müller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Cham, Switzerland: Springer, 2015, ch. 2, doi: 10.1007/978-3-319-21945-5.

[28] D. O'Shaughnessy, *Speech Communication: Human and Machine*. Reading, MA, USA: Addison-Wesley, 1987, p. 150.

[29] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016, chs. 6, 8, 9. [Online]. Available: https://www.deeplearningbook.org

[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[31] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, PMLR vol. 37, 2015, pp. 448–456. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.html

[32] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html

[33] D. W. Griffin and J. S. Lim, "Signal estimation from modified short-time Fourier transform," *IEEE Trans. Acoustics, Speech, and Signal Processing*, vol. 32, no. 2, pp. 236–243, Apr. 1984, doi: 10.1109/TASSP.1984.1164317.

[34] J. B. Allen and L. R. Rabiner, "A unified approach to short-time Fourier analysis and synthesis," *Proc. IEEE*, vol. 65, no. 11, pp. 1558–1564, Nov. 1977, doi: 10.1109/PROC.1977.10770.

Note on sources. Formulas in Chapter 2 come from the cited textbooks and papers: the STFT from [27], [34], the mel scale from [28], softmax, cross-entropy and gradient descent from [29], Adam from [30], FedAvg from [1] and FedProx from [4]. The calculations in Section 5.10 are my own derivations from these formulas. All figures and charts are my own work, drawn from my code and result files.

Appendices

Appendix A — Reproducing the Experiments

All commands are run from the repository root. See `docs/EXPERIMENTS.md` for details.

```
# broker
/opt/homebrew/sbin/mosquitto -c configs/mosquitto.conf

# baseline model (Section 5.1.2)
.venv/bin/python scripts/train_audio_cnn.py

# YOLO label check (Section 5.1.1)
.venv/bin/python scripts/fetch_vggsound_thumbnails.py
.venv/bin/python scripts/measure_label_noise.py

# strategies A, B and the C learning-rate sweep (Sections 5.2-5.4)
.venv/bin/python scripts/run_strategy.py --strategy a \
    --max-samples 5000 --stream-rate-ms 500 --num-rounds 10 \
    --buffer-size 500 --train-trigger-samples 1000
.venv/bin/python scripts/run_strategy.py --strategy b \
    --max-samples 5000 --num-rounds 10
for lr in 1e-3 5e-4 1e-4 5e-5; do
  .venv/bin/python scripts/run_strategy.py --strategy c \
    --max-samples 5000 --num-rounds 10 --epochs-per-round 1 \
    --local-lr $lr
done

# fusion, run 1 (Section 5.7)
.venv/bin/python scripts/spike_vggsound_pairs.py --per-class 200 \
    --max-attempts-per-class 600
.venv/bin/python scripts/prepare_vggsound_pairs.py
.venv/bin/python scripts/train_fusion.py --model all
.venv/bin/python scripts/train_fusion.py --model fusion \
    --modality-dropout 0.3
.venv/bin/python scripts/train_fusion.py --model fusion \
    --modality-dropout 0.5
.venv/bin/python scripts/train_fusion.py --model fusion \
    --modality-dropout 0.5 --audio-warmstart
.venv/bin/python scripts/eval_fusion_blackout.py

# fusion, run 2 (Section 5.7)
.venv/bin/python scripts/train_fusion.py --model fusion --epochs 25 \
    --es-patience 25
.venv/bin/python scripts/train_fusion.py --model fusion \
    --modality-dropout 0.5 --epochs 25 --es-patience 25
.venv/bin/python scripts/train_fusion.py --model fusion \
    --modality-dropout 0.5 --epochs 25 --es-patience 25 \
    --audio-warmstart
.venv/bin/python scripts/eval_fusion_blackout.py
.venv/bin/python scripts/probe_fusion_audio_use.py

# FB and FC (Section 5.8)
.venv/bin/python scripts/prepare_fusion_fv.py
.venv/bin/python scripts/run_strategy.py --strategy fb \
    --buffer-size 250 --train-trigger-samples 500 --num-rounds 6 \
    --stream-rate-ms 50
.venv/bin/python scripts/run_strategy.py --strategy fc --num-rounds 10 \
    --local-lr 1e-4

# live data: training-settings sweep (Section 5.6.3) and latency (Section 5.9)
.venv/bin/python scripts/sweep_live_training.py
.venv/bin/python scripts/measure_inference_latency.py

# all result charts of Chapter 5
```

```
.venv/bin/python scripts/make_thesis_figures.py
```

The live Raspberry Pi runs follow the step-by-step runbook in `docs/PRESENTATION.md`. Each strategy run writes `server.json`, `client_A.json`, `client_B.json`, logs and an `invocation.txt` with the exact command to `results/strategy_<x>/run_<time>/`.

Appendix B — Full Per-Round Tables

All values are copied from the `server.json` file of each run.

Table B.1: Strategy A, centralized.

Round	Test accuracy	Server training time
1	0.638	3.56 s
2	0.681	7.27 s
3	0.644	10.00 s
4	0.638	12.23 s
5	0.612	14.51 s
6	0.591	18.17 s
7	0.639	21.96 s
8	0.664	23.11 s
9	0.671	26.32 s
10	0.661	30.12 s

Source: `results/strategy_a/run_20261001_160849/server.json`.

Table B.2: Strategy B, hybrid.

Round	Test accuracy	Server training time
1	0.621	3.74 s
2	0.634	5.60 s
3	0.664	8.31 s
4	0.665	11.27 s
5	0.654	13.85 s
6	0.698	17.06 s
7	0.679	19.95 s
8	0.675	22.06 s
9	0.685	26.10 s
10	0.688	30.12 s

Source: `results/strategy_b/run_20261001_165113/server.json`.

Table B.3: Strategy C, federated, local learning rate 1e-3.

Round	Test accuracy	Server averaging time
1	0.422	0.54 s
2	0.504	0.58 s
3	0.440	0.55 s
4	0.526	0.57 s
5	0.570	0.51 s
6	0.600	0.54 s
7	0.564	0.61 s
8	0.659	0.57 s
9	0.559	0.63 s
10	0.554	0.63 s

Source: `results/strategy_c/run_20261001_180949/server.json`.

Table B.4: Strategy C, federated, local learning rate 5e-4.

Round	Test accuracy	Server averaging time
1	0.633	0.59 s
2	0.695	0.64 s
3	0.676	0.62 s
4	0.545	0.67 s
5	0.539	0.65 s
6	0.595	0.68 s
7	0.606	0.78 s
8	0.514	0.65 s
9	0.583	0.66 s
10	0.607	0.68 s

Source: `results/strategy_c/run_20261001_182234/server.json`.

Table B.5: Strategy C, federated, local learning rate 1e-4.

Round	Test accuracy	Server averaging time
1	0.675	0.64 s
2	0.624	0.64 s
3	0.665	0.65 s
4	0.689	0.83 s
5	0.692	0.80 s
6	0.705	0.73 s
7	0.674	0.78 s
8	0.640	0.87 s
9	0.673	1.23 s
10	0.699	0.75 s

Source: `results/strategy_c/run_20261001_182706/server.json`.

Table B.6: Strategy C, federated, local learning rate 5e-5.

Round	Test accuracy	Server averaging time
1	0.645	0.69 s
2	0.668	0.66 s
3	0.694	0.66 s
4	0.679	0.69 s
5	0.694	0.72 s
6	0.662	0.70 s
7	0.674	0.71 s
8	0.671	0.70 s
9	0.663	0.74 s
10	0.676	0.72 s

Source: `results/strategy_c/run_20261001_183224/server.json`.

Table B.7: FB, hybrid fusion.

Round	Clean accuracy	Blackout accuracy	Server time
1	0.709	0.485	2.80 s
2	0.670	0.515	4.04 s
3	0.670	0.505	4.36 s
4	0.748	0.602	6.08 s
5	0.748	0.553	7.76 s
6	0.845	0.583	9.18 s

Source: `results/strategy_fb/run_20261001_193556/server.json`.

Table B.8: FC, federated fusion.

Round	Clean accuracy	Blackout accuracy	Server time
1	0.786	0.476	0.27 s
2	0.786	0.427	0.24 s
3	0.816	0.563	0.28 s
4	0.854	0.612	0.24 s
5	0.854	0.583	0.24 s
6	0.845	0.583	0.25 s
7	0.874	0.670	0.21 s
8	0.854	0.621	0.26 s
9	0.883	0.602	0.24 s
10	0.854	0.573	0.23 s

Source: `results/strategy_fc/run_20261001_193733/server.json`.

Appendix C — Evidence Gallery Samples

Figure 5.4 shows four frames from the enrollment evidence gallery. The full gallery, with the frame, the 500 ms of audio and the label for every labelled tick, is in `data/pi_captures/results/live/evidence_20260903_204117/`.

Appendix D — Source Code

https://github.com/adnananwarrajput6034-blip/edgeComputing